香港中文大學
The Chinese University of Hong Kong

# ROSE5710 Fundamental Robotics and Systems

# Lecture Set 2: Robot Hardware and Software

*Darwin Lau*

*2026-27*

# Lecture Summary

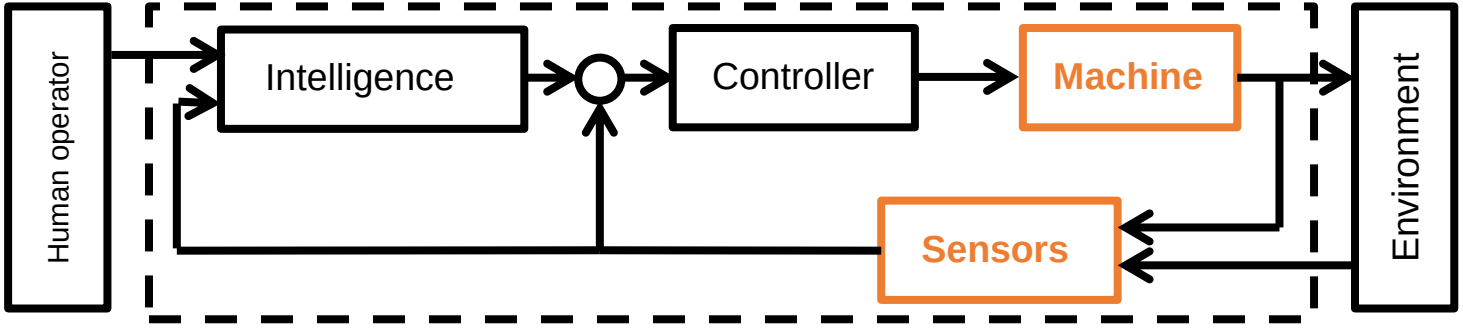
## Aims of the lecture

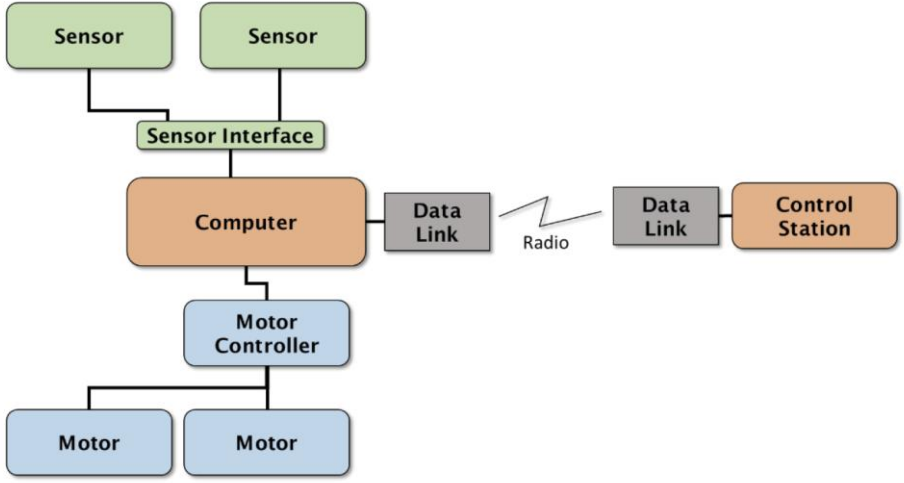
- Introduce common robot sensors and actuators
- Introduce the role of mechanisms in robotics
- Introduce the mechatronics architecture in robotics
- Introduce ROS2

## References


- Intelligence for Robotics, 2nd edition, Francis X. Govers III, Packt Publishing, 2024
- Introduction to Autonomous Robots: Mechanisms, Sensors, Actuators, and Algorithms, Nikolaus Correll, Bradley Hayes, Christoffer Heckman, and Alessandro Roncone, MIT Press, 2022
- https://docs.ros.org/

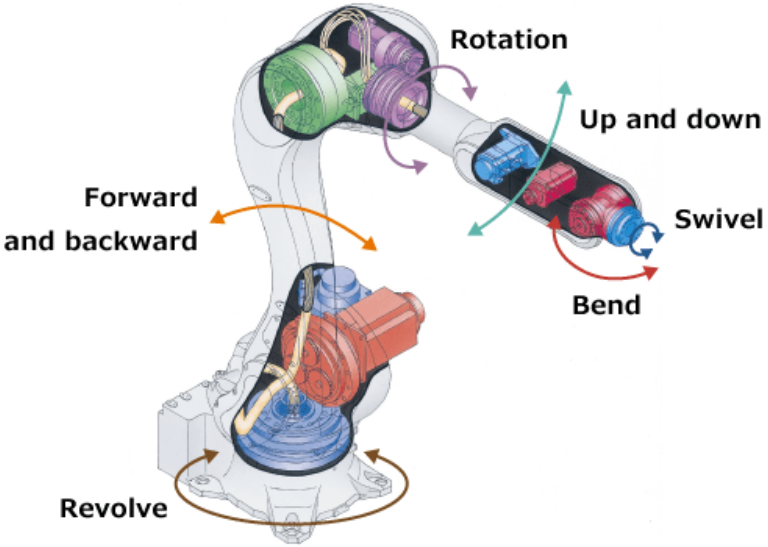
# Robot Anatomy

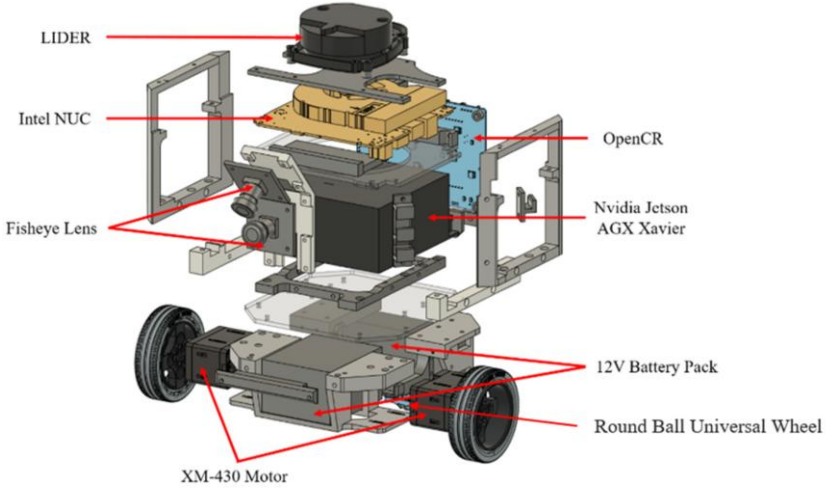


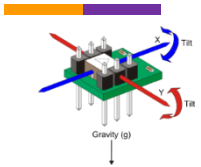
# Robot Anatomy

The components of a robot



# The Purposes of Sensors



Accelerometer


Gyro


Pendulum Resistive Tilt Sensors


Piezo Bend Sensor

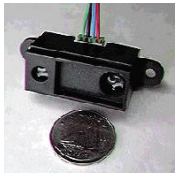

Metal Detector



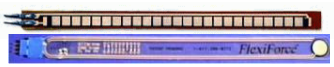
Gas Sensor


Gieger-Muller Radiation Sensor


Pyroelectric Detector

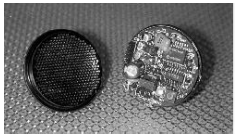

Digital Infrared Ranging


CDS Cell
Resistive Light Sensor



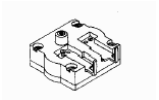
Resistive Bend Sensors

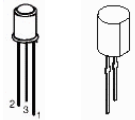

UV Detector


Miniature Polaroid Sensor


Mechanical Tilt Sensors


Touch Switch


Pressure Switch


IR Pin Diode



IR Sensor w/lens


Limit Switch


Thyristor


Hall Effect Magnetic Field Sensors


Stereo Vision


IR Reflection Sensor


IR Amplifier Sensor


IRDA Transceiver

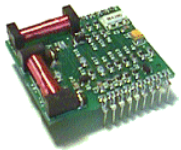

Magnetic Sensor


Load cell

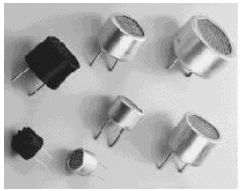

Radio Shack Remote Receiver


IR Modulator Receiver


Solar Cell


Compass


Encoder


Piezo Ultrasonic Transducers

# Classification of Sensors

## What the sensors aim to measure?

- **Internal sensors (proprioceptive):** Internal state of the robot
- **External sensors (exteroceptive):** Sense outside the physical embodiment of the robot

## Internal sensors

- Includes robot's joint angles, its speeds, as well as internal torques and forces

## External sensors

- Important because it is crucial for the robot to correctly perceive the state of the world

# The Purposes of External Sensors

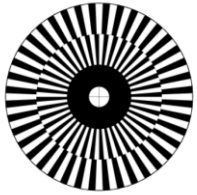


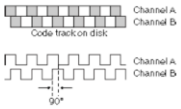

Andrea Cherubini and David Navarro-Alarcon, “Sensor-Based Control for Collaborative Robots: Fundamentals, Challenges, and Opportunities”, *Frontiers in Neurorobotics*, 2021

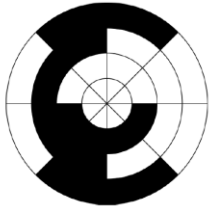
# Common Sensor: Angle Sensor

Most common type of sensor in robotics



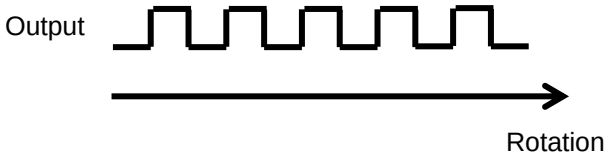




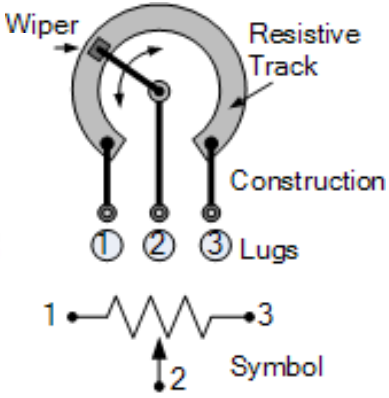
**Relative encoder**

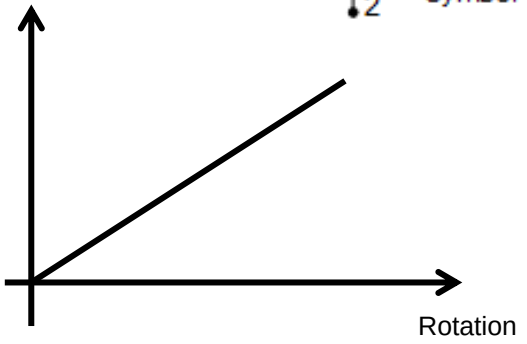
**Grey code absolute encoder**




**Optical encoder**



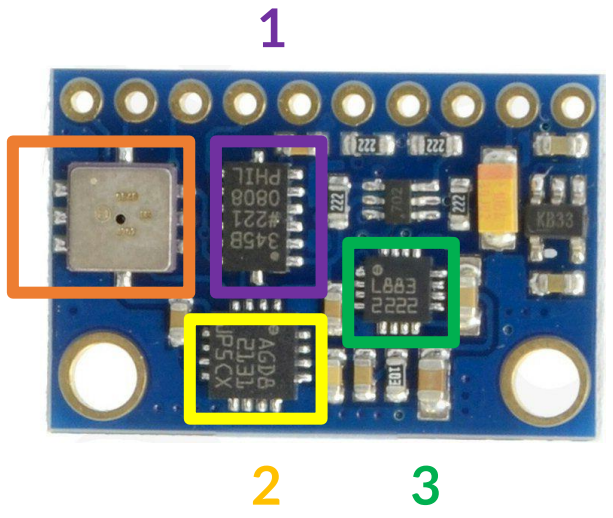



**Potentiometer**

## Internal sensor

# Common Sensor: IMU




A combination of sensors to measure the “inertia”

1. 3-axis accelerometer
2. 3-axis gyroscope
3. 3-axis digital compass
4. Barometric pressure and temperature sensor

Provides 6 degrees of freedom (DoF) information of robot

- Orientation is more reliable than translation for an IMU

**Internal sensor**

# Common Sensor: IMU



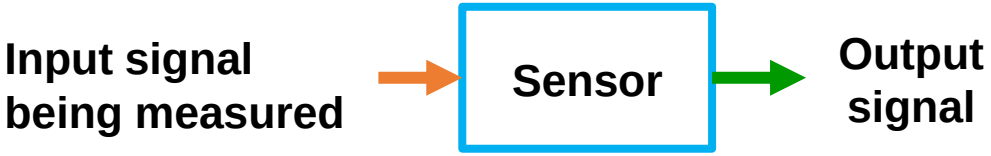
# Sensor Input-Output and Calibration

## Sensor input

- The type of signal being entered to be measured
- Example for encoder: rotation angle

## Sensor output

- What the sensor outputs for further processing
- Example for encoder: encoder count



# Sensor Properties: Linearity, Sensitivity and Resolution

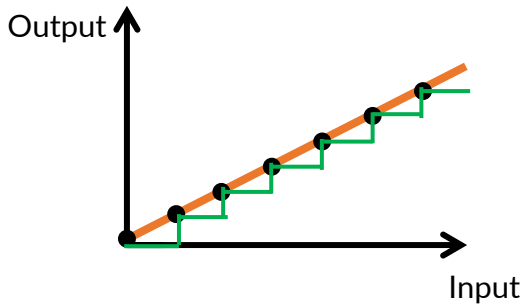
**Linearity**: input-output relationship

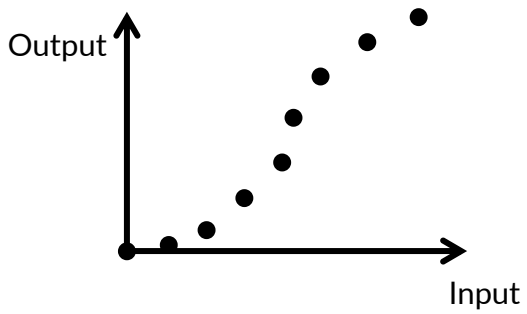
**Sensitivity:** amount of output for unit input

**Resolution**

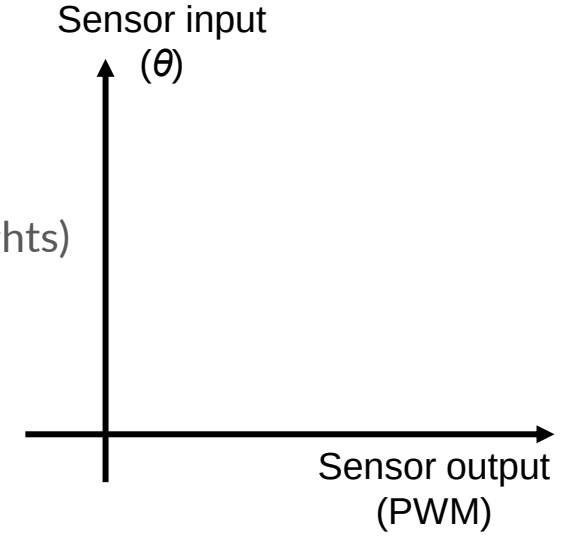
- The smallest change in angle that could be detected

$$R = \frac{\Delta \text{input}}{\Delta \text{output}}$$



# Sensor Calibration




Process to determine the input-output relationship

1. Provide known sensor input manually (angles/distances/weights)
2. Read sensor output
3. Plot the output vs input graph
4. Using linear fitting, compute the equation for each sensor

$$\theta_i = S_i \times (PWM - O_i)$$

What is the meaning of $S_i$ and $O_i$ ?

# Sensor Properties: Accuracy and Precision

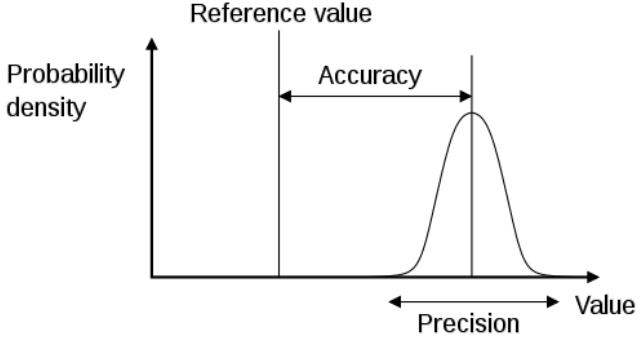
## Accuracy:

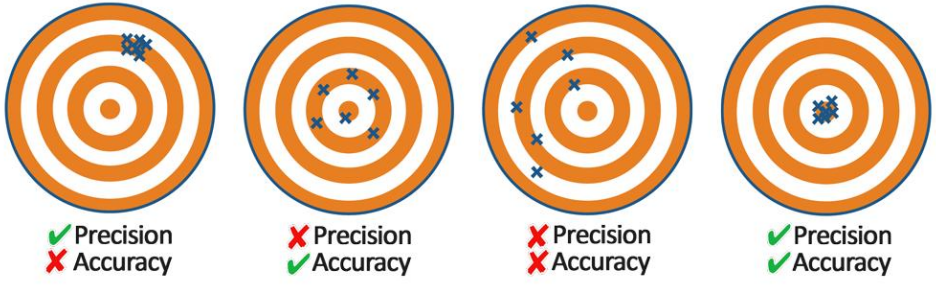
- Difference between the measured value and true value (*m* average output, *v* true value)

$$accuracy = 1 - \frac{|m - v|}{v}$$

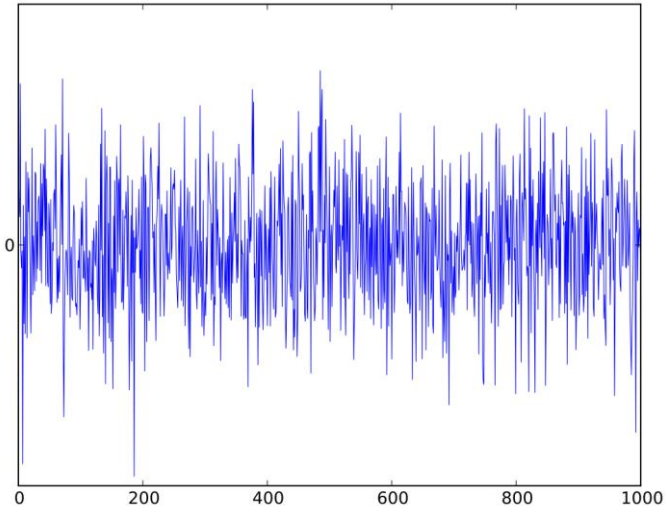
## Precision:

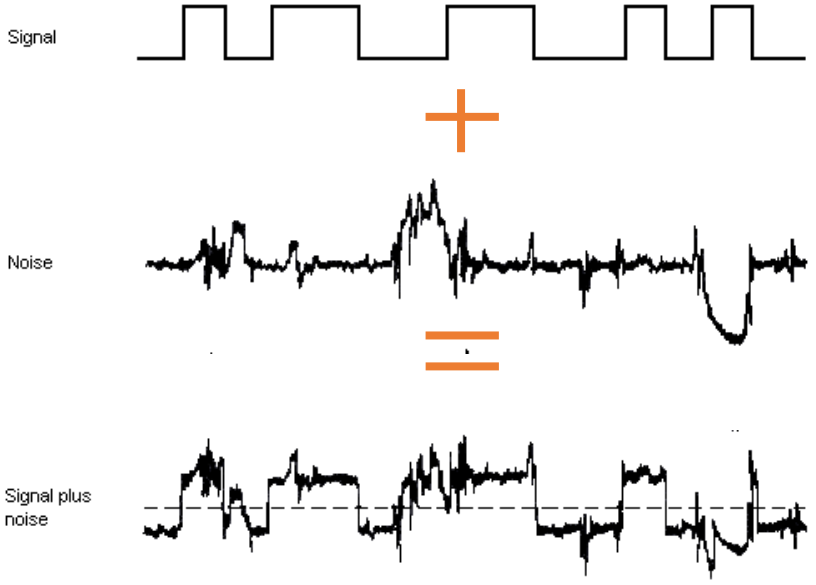
- Repeatability of measurement



# Noise and De-Noise



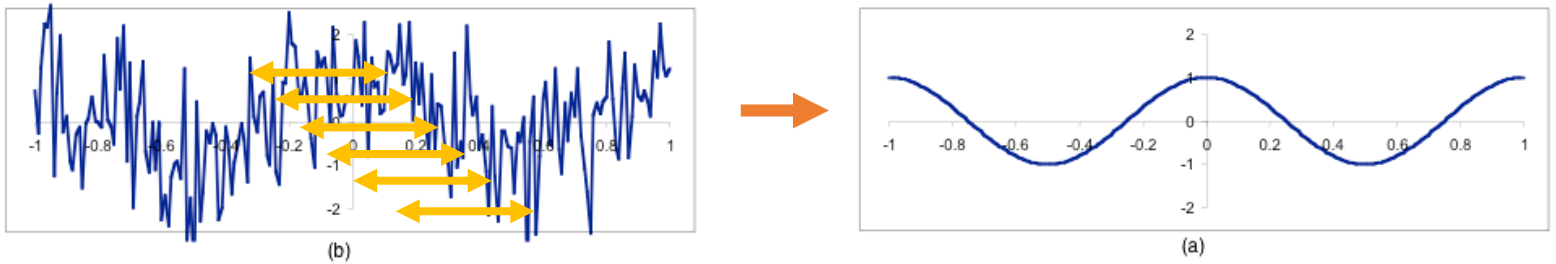



# Noise and De-Noise

## Moving average filter

- A process to make noisy signal cleaner to use
- At instance k with a window of N samples

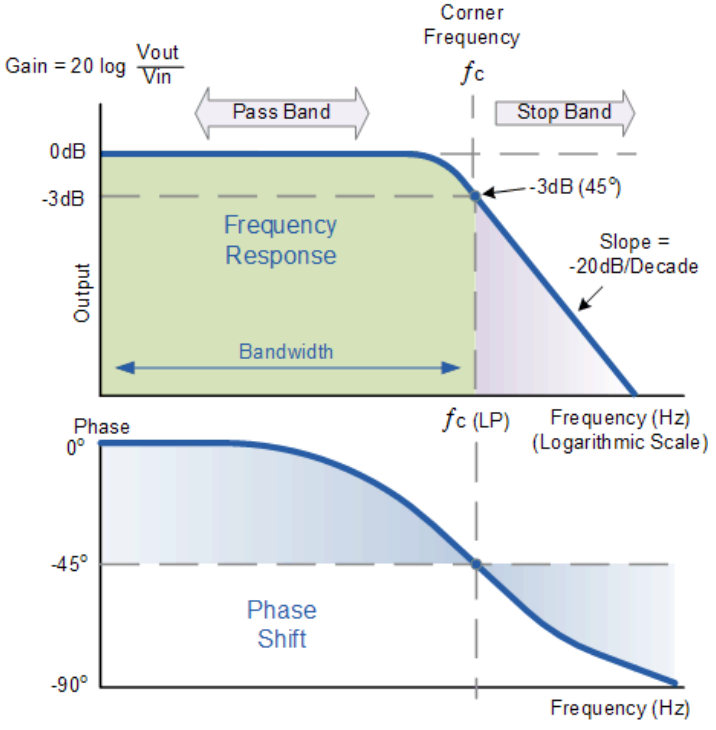
$$x_k = \frac{1}{N}\sum_{i=0}^{N-1} x_{k-i}$$



# Noise and De-Noise

## Low-pass filter

- Operates based on frequency
- Let low frequency pass through (robot motion)
- But attenuate higher frequency (noise)




## The selection of cut-off frequency is critical

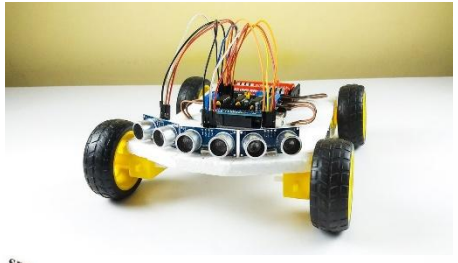
- Attenuation of signal
- Phase shift causes lag
- Hence best if motion and noise frequency are well separated

# Common Sensor: Ultrasonic Sensors

Sends a ultrasonic wave and waits for the rebound

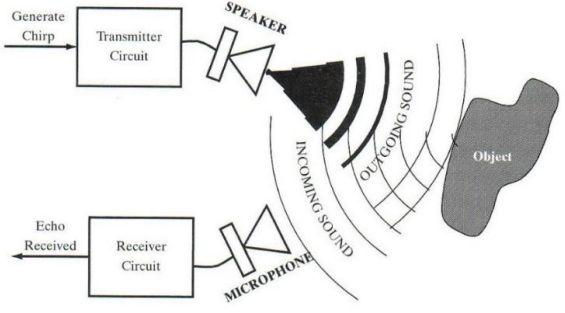
1. Emit a quick burst of ultrasound (40 - 180kHz)
2. Measure the elapsed time until the receiver indicates that an echo is detected.
3. Determine how far away the nearest object is from the sensor

Used heavily for obstacle avoidance



Problems

- Delay due to the speed of sound
- Senses over a “cone range”






**External sensor**

# Common Sensor: Laser Range Finder

Same idea as ultrasonic sensor, except uses **light** instead

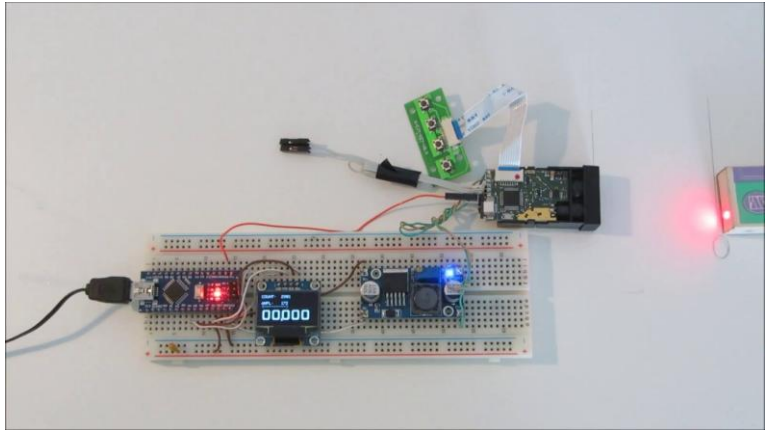
- Much faster than ultrasonic sensor
- For 3m distance, it takes about 10 milliseconds for an ultrasonic system but only 10 nanoseconds for laser (speed of light vs sound)

Used to measure distance accurately

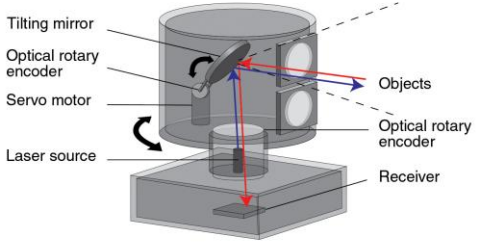
Problems

- Measuring the “time of flight”
- Variation in propagation speed

**External sensor**



# Common Sensor: LiDAR and SLAM



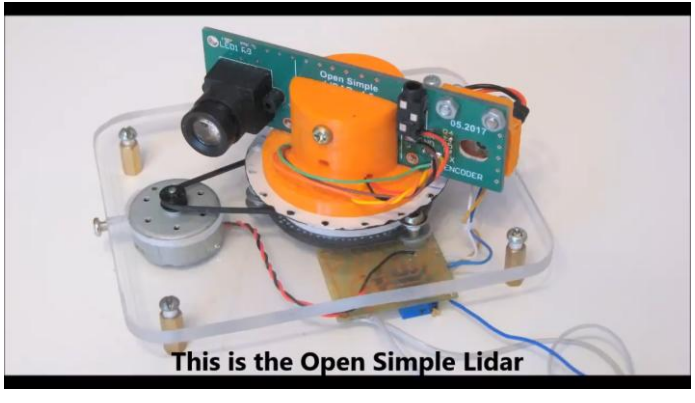

## LiDAR (Light Detection and Ranging)

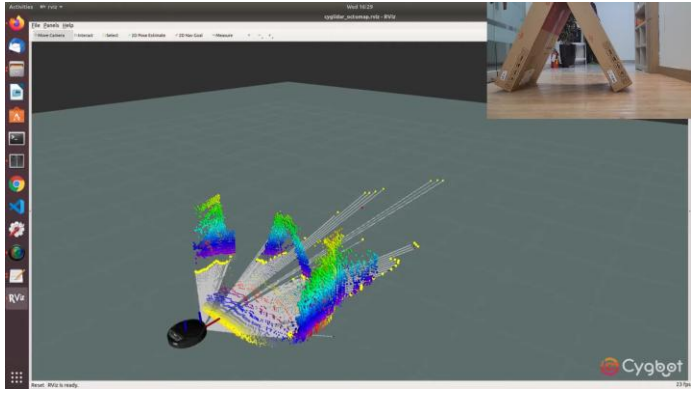
- Typically a rotating laser to get 360° information
- 2D and 3D LiDAR available

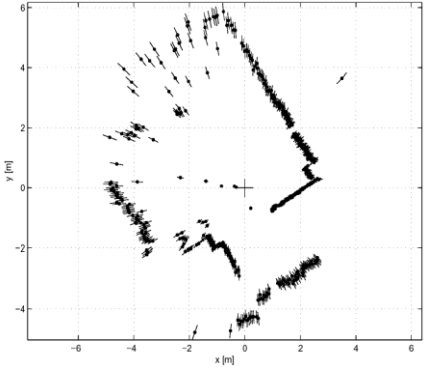


## SLAM

- Simultaneous Localisation And Mapping



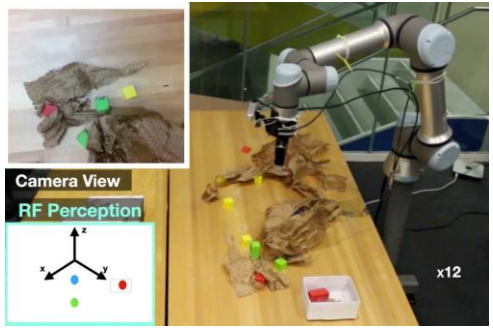


# Common Sensor: Vision



Vision is very difficult and also very important in robotics

- Geometric information
- Texture
- Color etc.




Many applications in robotics

- Distance measurement
- Object/person recognition
- Control

Vision systems: single camera, stereo (3D), etc.

# Common Sensor: Vision

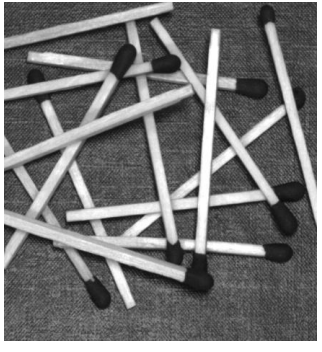
Red Green Blue Depth



# Common Sensor: Vision

## Challenges in vision





Segmentation











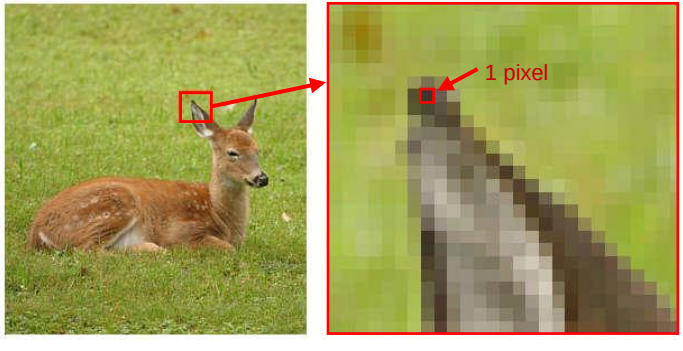




Classification



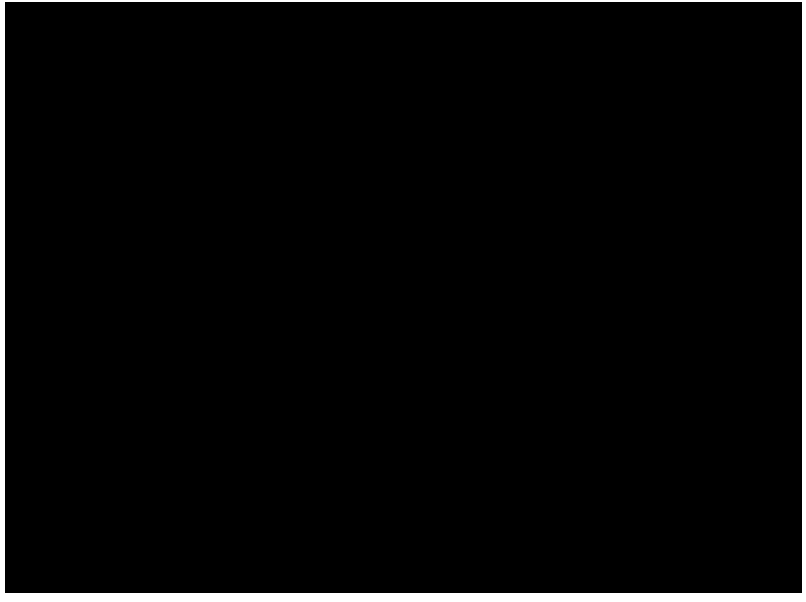
Optical illusion

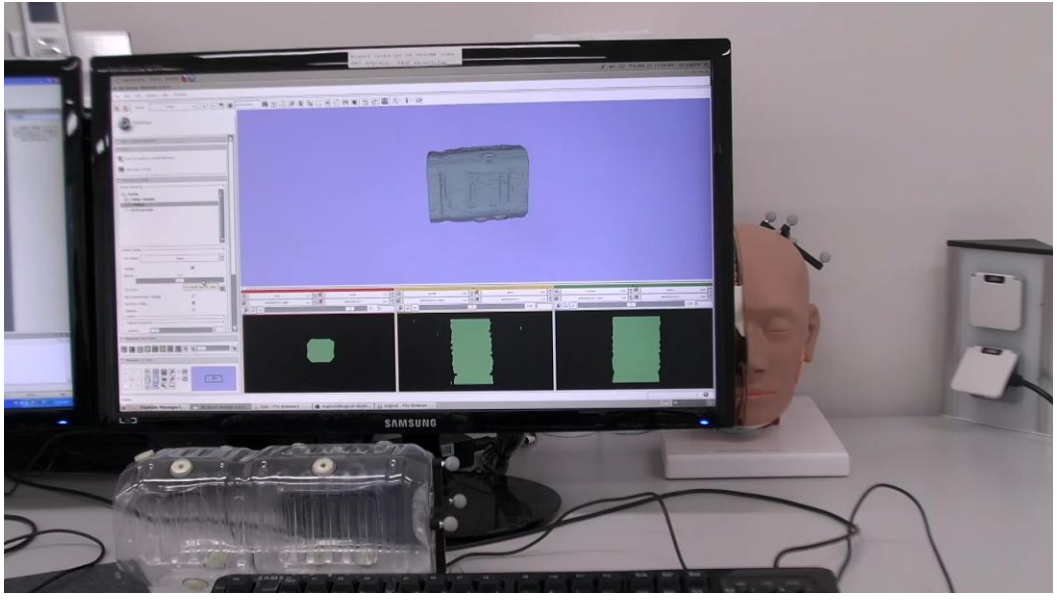



Resolution

# Common Sensor: Motion Tracking

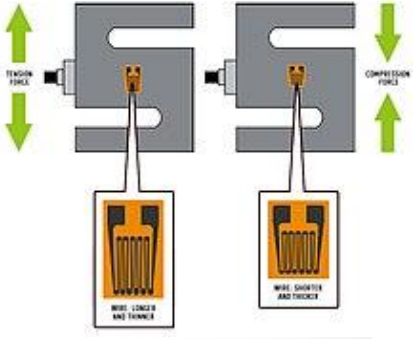
"Vision" with infrared tracking balls



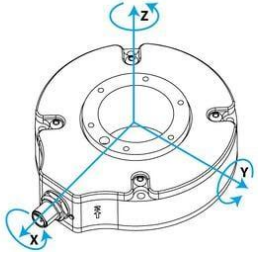


# Common Sensor: Force Sensor

Force sensing is important for a range of robot manipulation applications







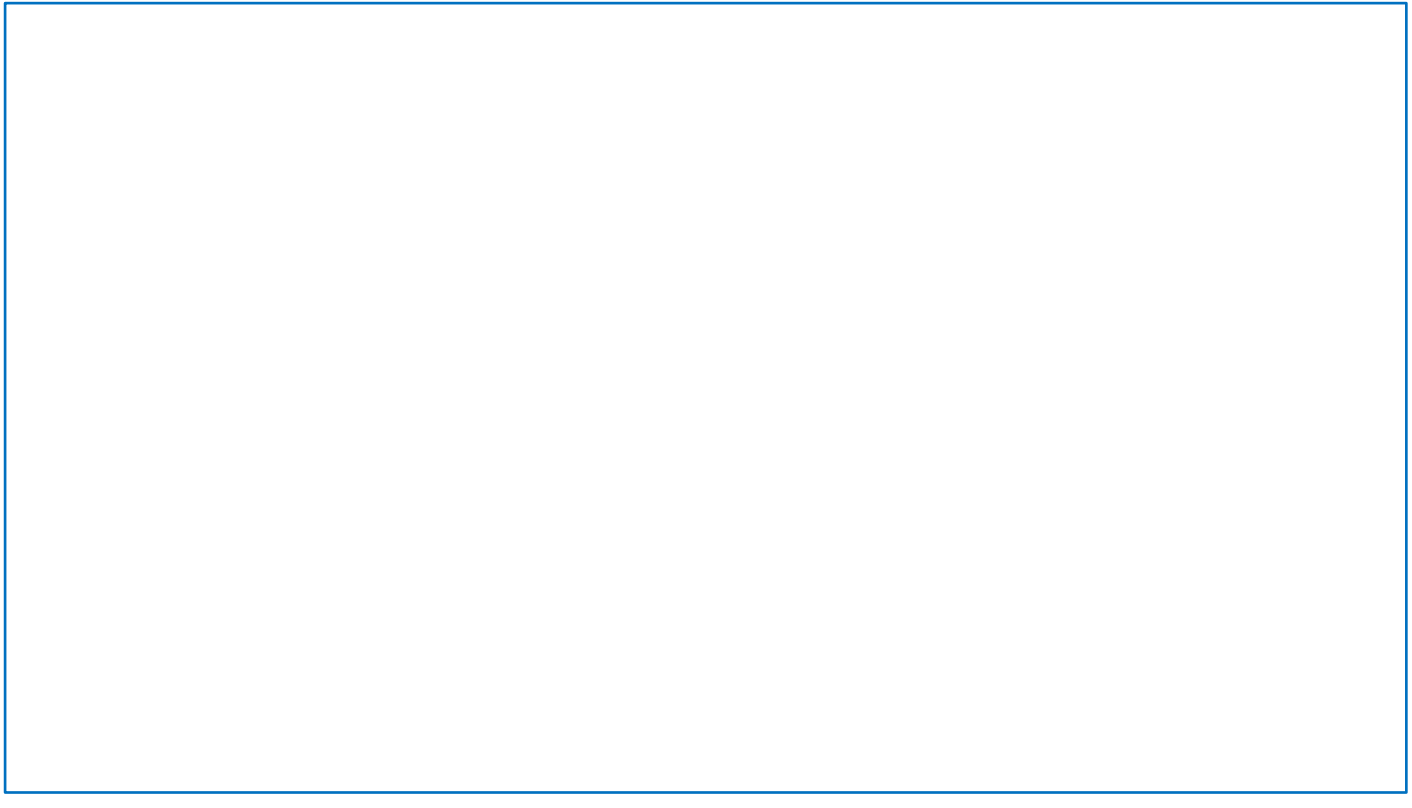

# Common Sensor: Force Sensor

Force sensing plus control

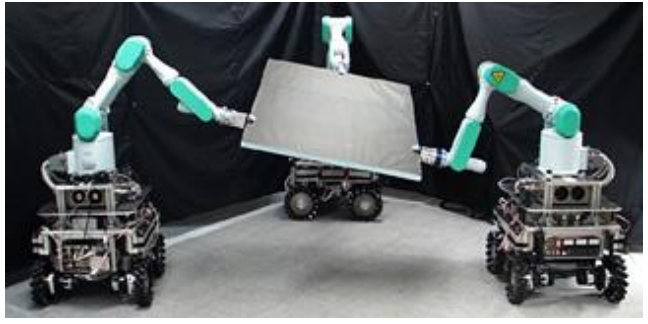


# Common Sensor: Force Sensor

## Artificial Skin



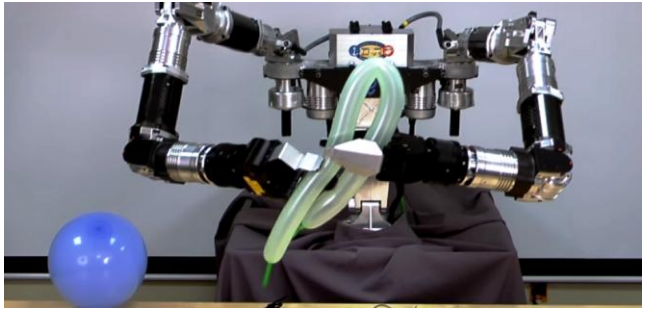
# Common Actuators









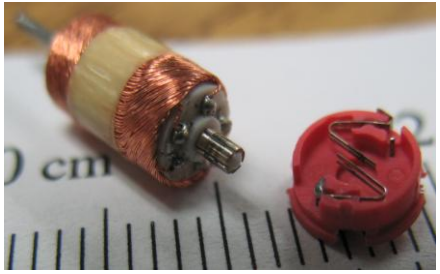


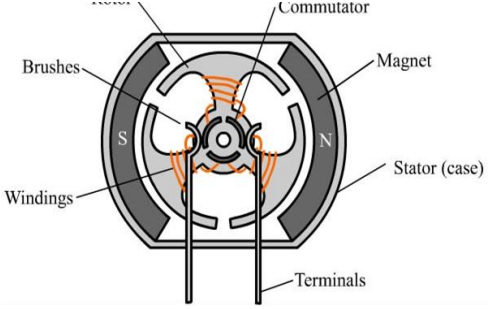





# Common Actuator: Electric Motors

Converts electric energy into mechanical energy





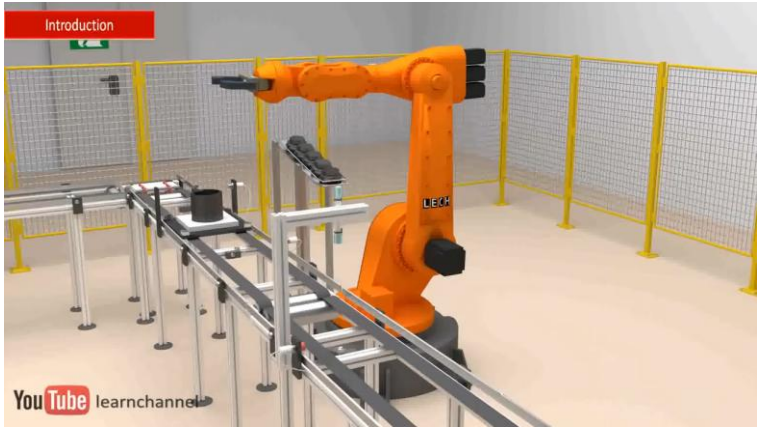



- Easy to control
- Accurate
- Fast response
- Clean
- Many different types of electric motors exist
- Low-power output compared with some other actuators

# Common Actuator: DC Servo Motors

Very common type of motor in robotics

- AC and DC (but mostly DC for robots)
- Many types and specifications
- Controlled using PWM
- Control position/velocity/acceleration











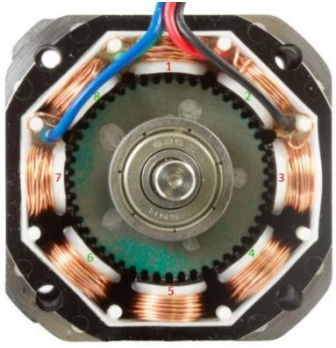
Source: https://www.youtube.com/watch?v=hg3TIFIxWCo

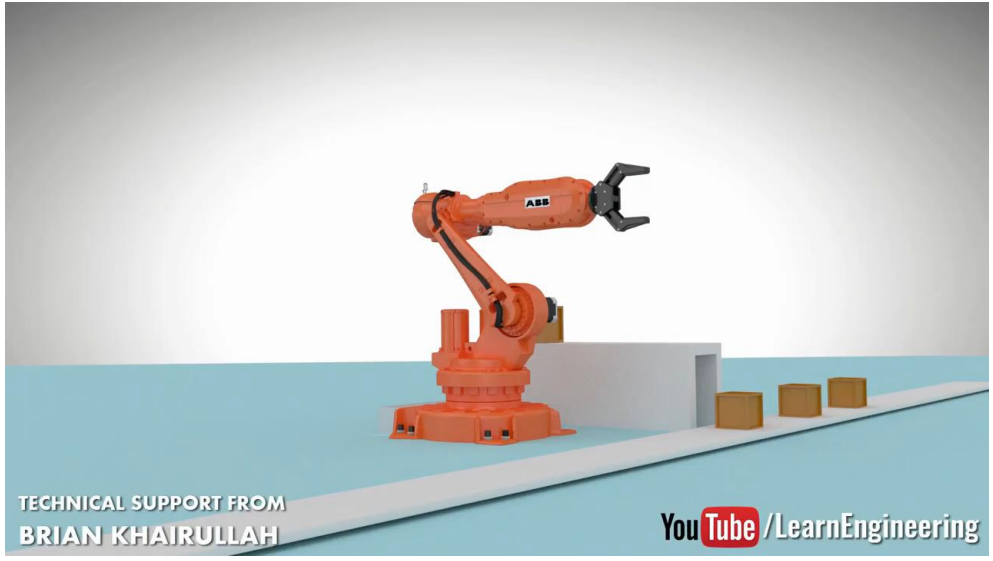
# Common Actuator: DC Stepper Motor

Stepper motors are a variant of DC motors

- Focused on high position accuracy
- Weaker in torque/power







# Common Actuator: RC Servo Motor

## Radio Control Servo

- Simple and easy to use
- Low in cost
- Position command only (mostly)



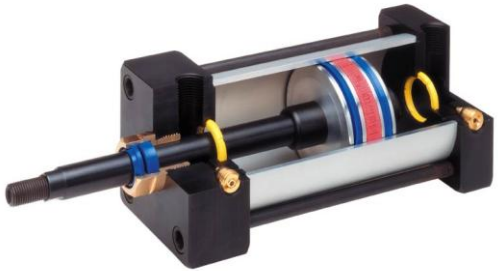
- Great for testing/prototyping
- Or lightweight applications
- And low precision requirements

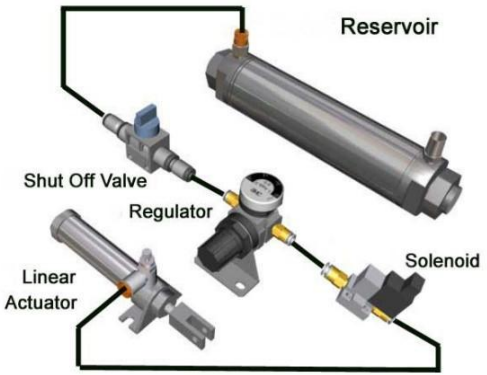


# Common Actuator: Pneumatics

Name from Greek πνευμα (pneuma), means “wind, air, breath, spirit”

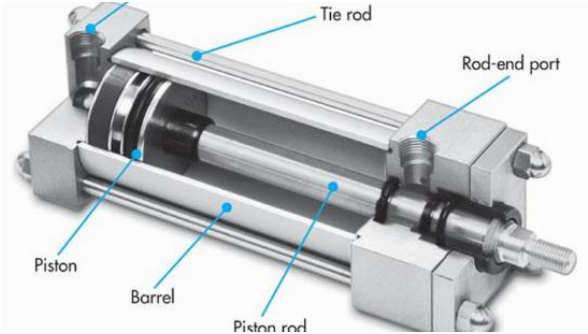
- Converts compressed air into mechanical motion
- Clean in using air
- Not so precise
- Good power
- Flexible and soft robots







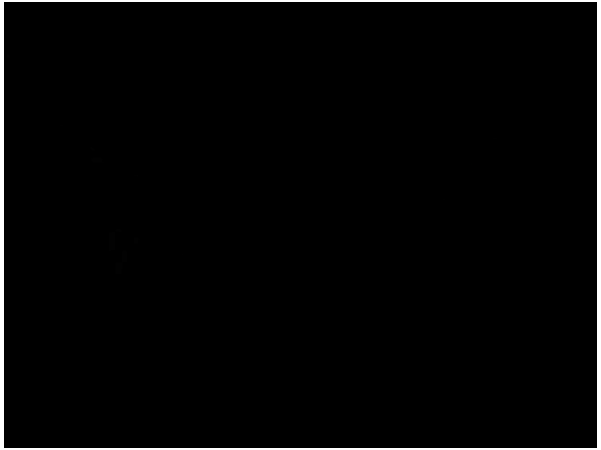

# Common Actuator: Pneumatics








**Pneumatic Pistons:**
**https://www.youtube.com/watch?v=Rsqc1-vjkRs**

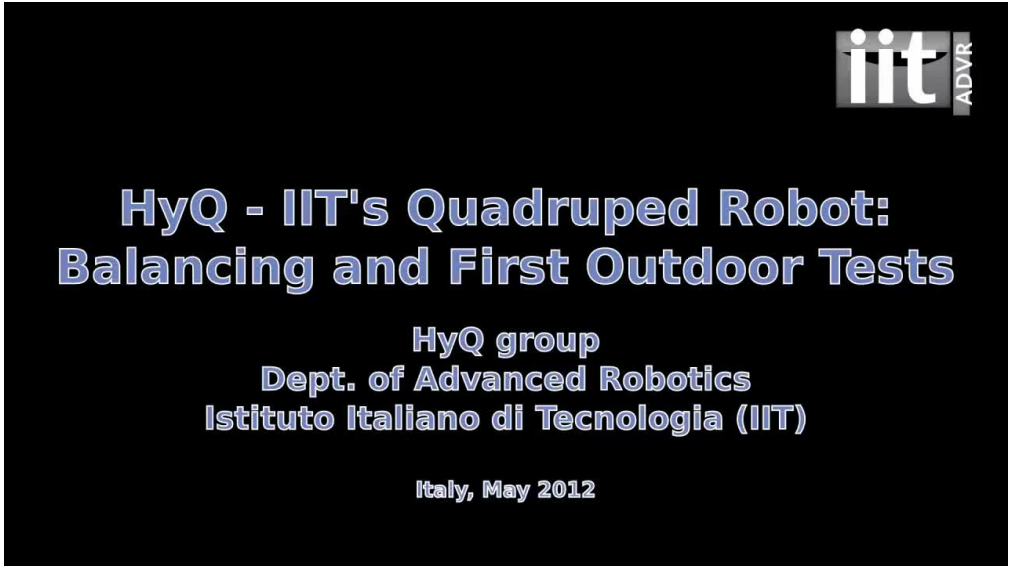


**FESTO Muscle Robot**
**https://www.youtube.com/watch?v=2iG1ybuchx0**

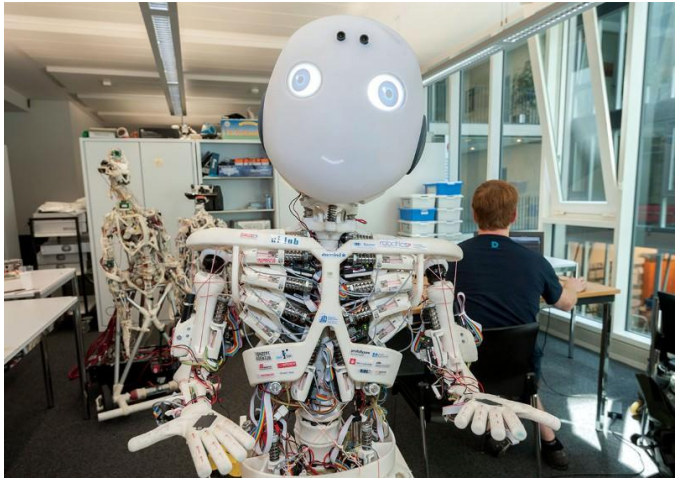
# Common Actuator: Hydraulics

Similar to the idea of pneumatics, but using oil or fluid

- Higher strength and more smooth
- The liquid can be very messy
- Typically larger/heavier

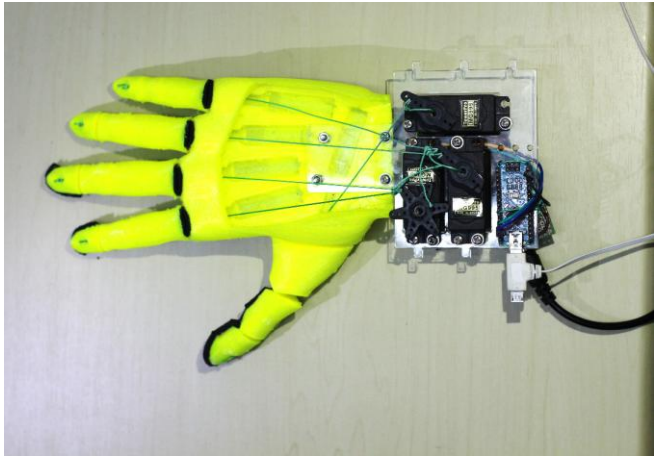


# Cable-Driven Robots (Transmission)



Use cables to relocate the location of motors

- Large distance
- Lightweight system
- Place motors at more desirable locations
- Simple dynamics





香港中文大學
The Chinese University of Hong Kong

# Robots are Different Because of Mechanisms



Serial Robots



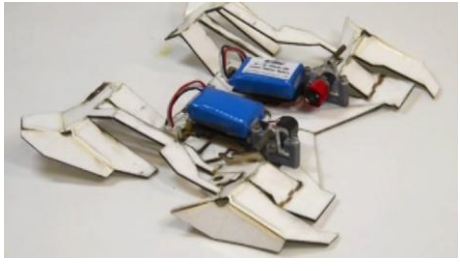
Parallel Robots



Unmanned Ground Robots



Unmanned Aerial Robots



Origami Robots



Robot Hands



Quadruped Robots




Humanoid Robots

香港中文大學
The Chinese University of Hong Kong

# Wheels and Mobile Robots

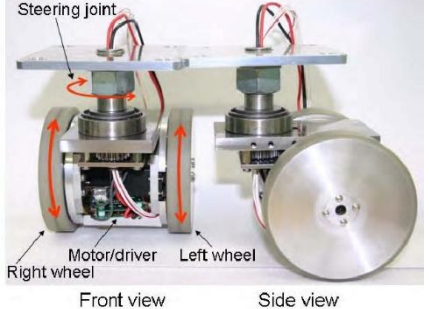
## Mobile bases

- Two wheels (differential drive)
- Omni-wheel (can be 3 or 4 wheels) and mecanum wheels
- Active casters





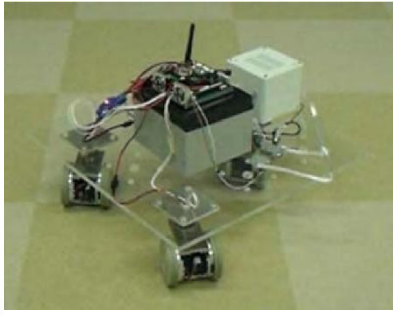










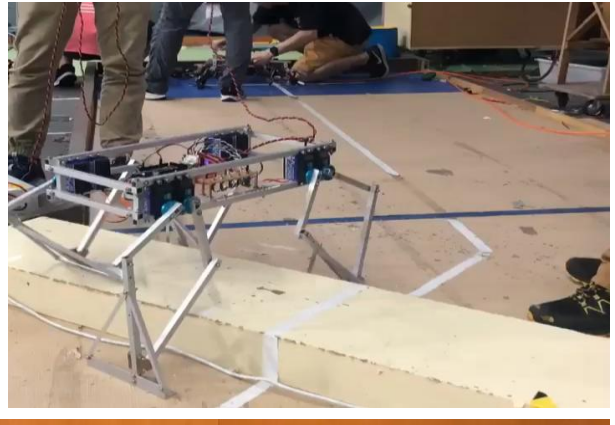
# Wheels and Mobile Robots

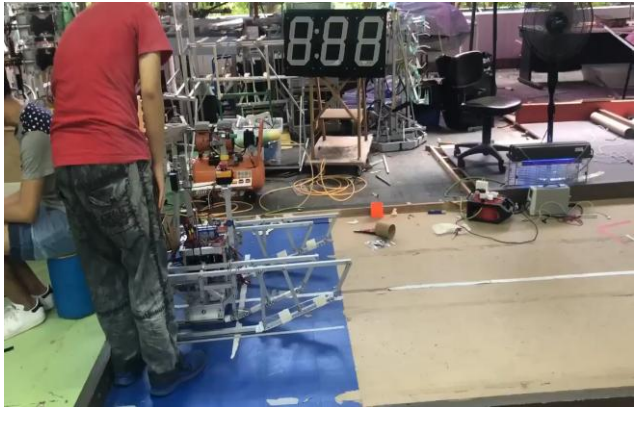
## Omni-directional motion

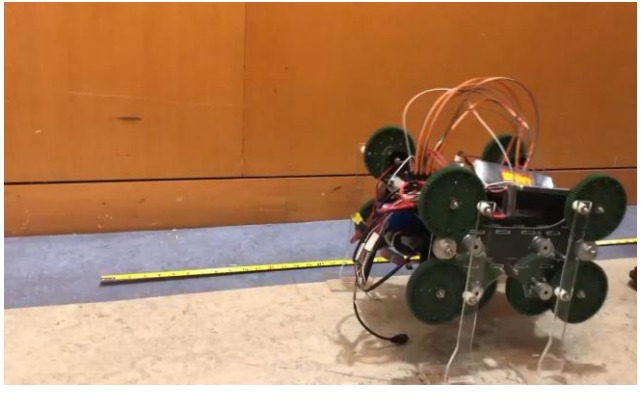


# Legged Robots

4 legs can have many “variations”



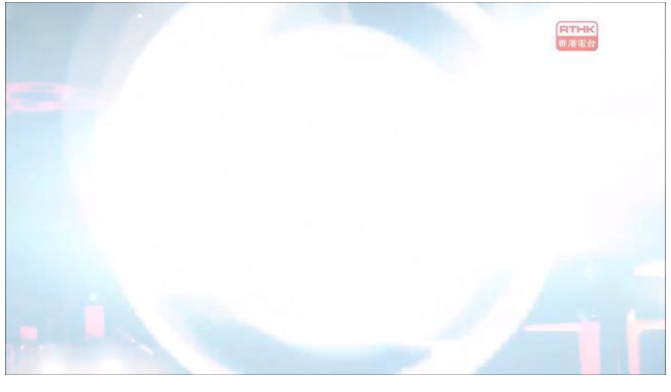




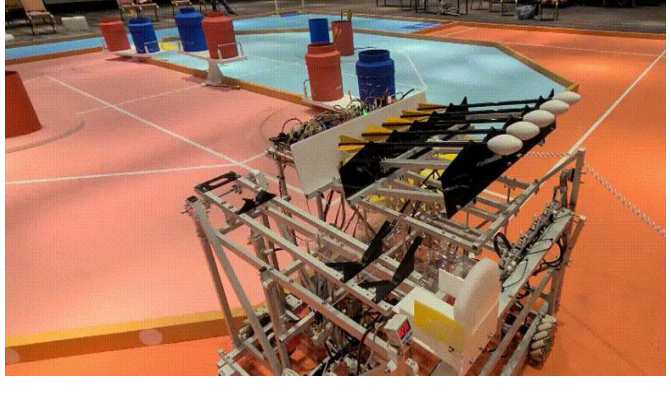


# Throwing and Shooting

Different was to “shoot”



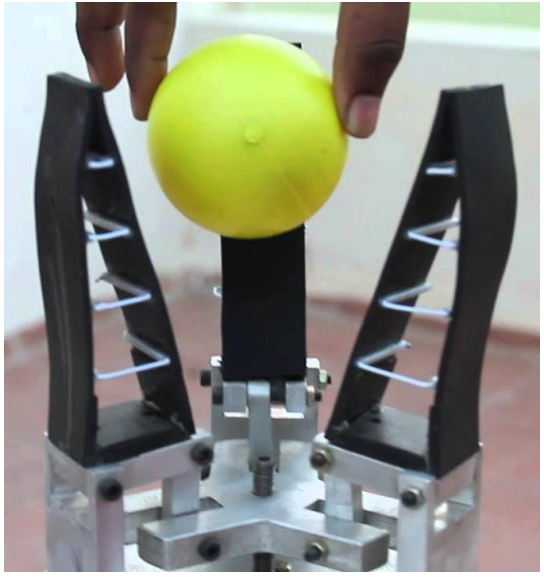




# Grasping

## Flexible gripper

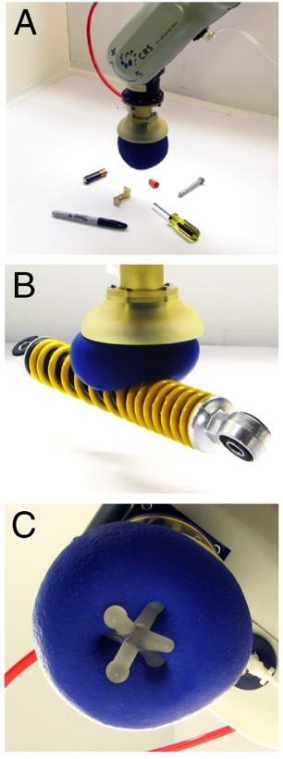


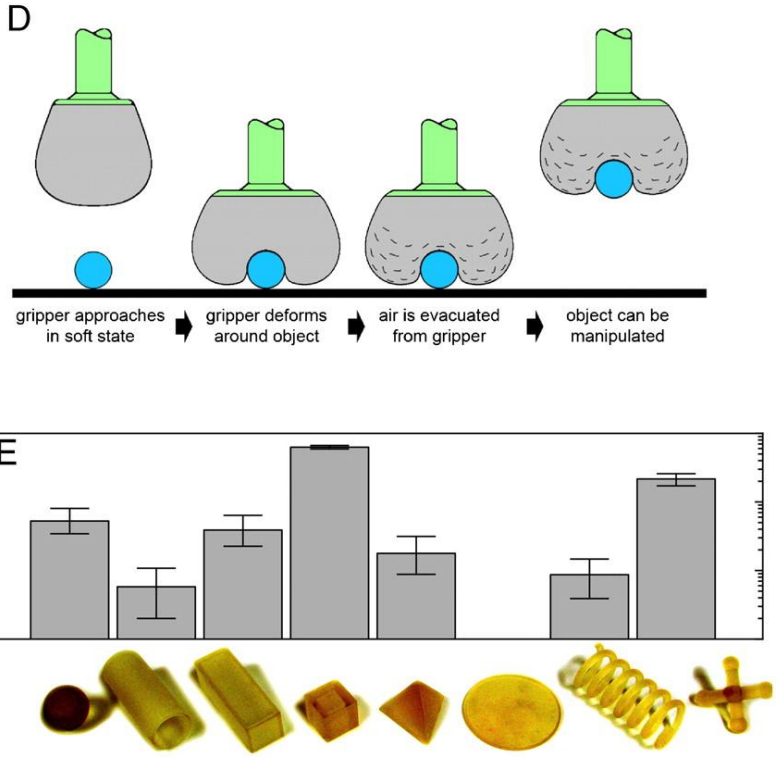


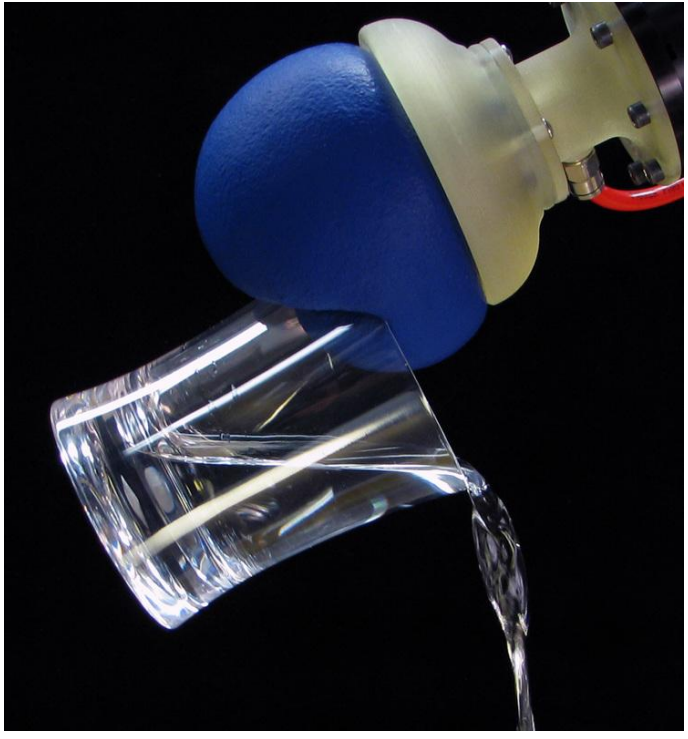
香港中文大學
The Chinese University of Hong Kong

# Grasping

## Universal gripper



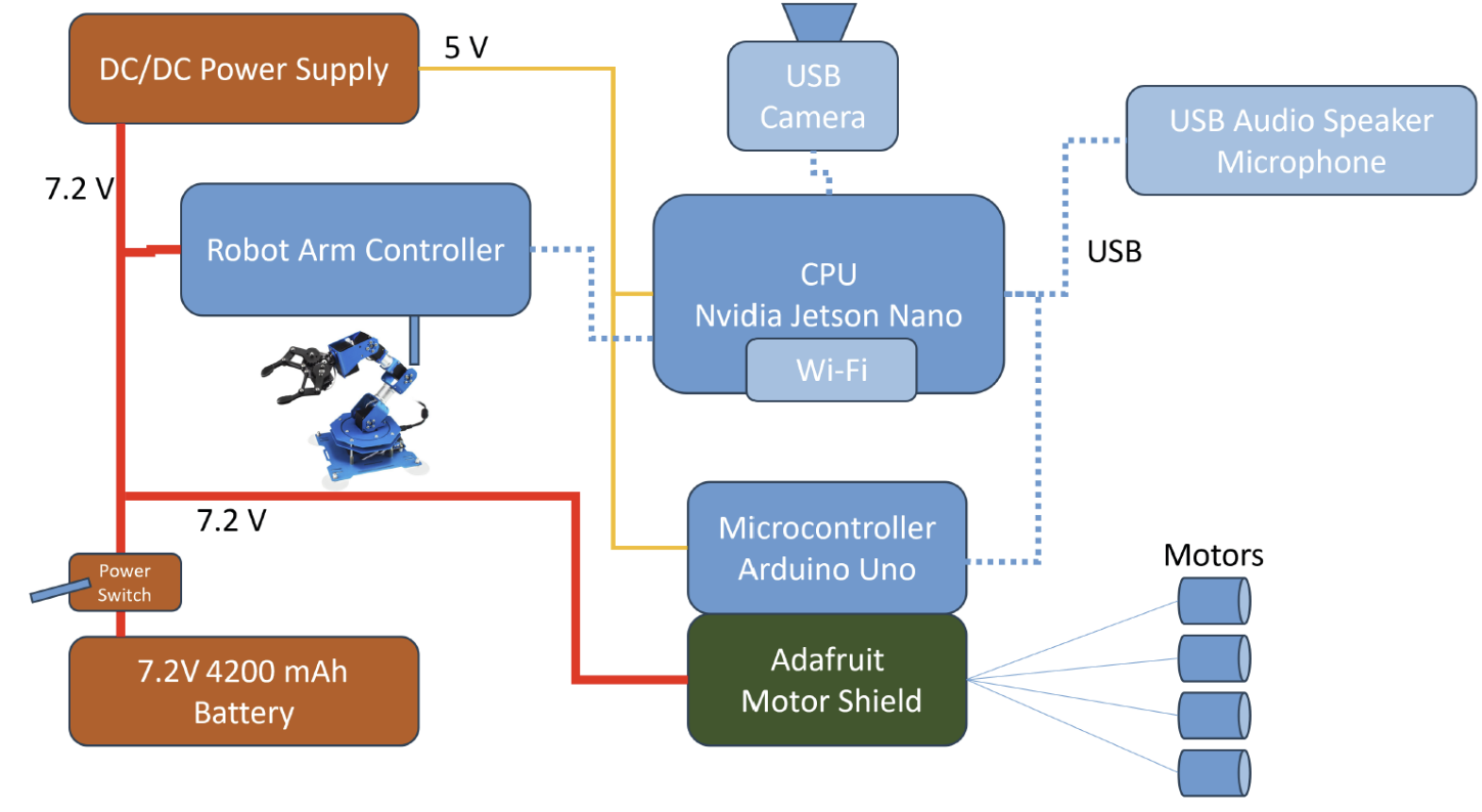
# Grasping

## Pneumatic gripper

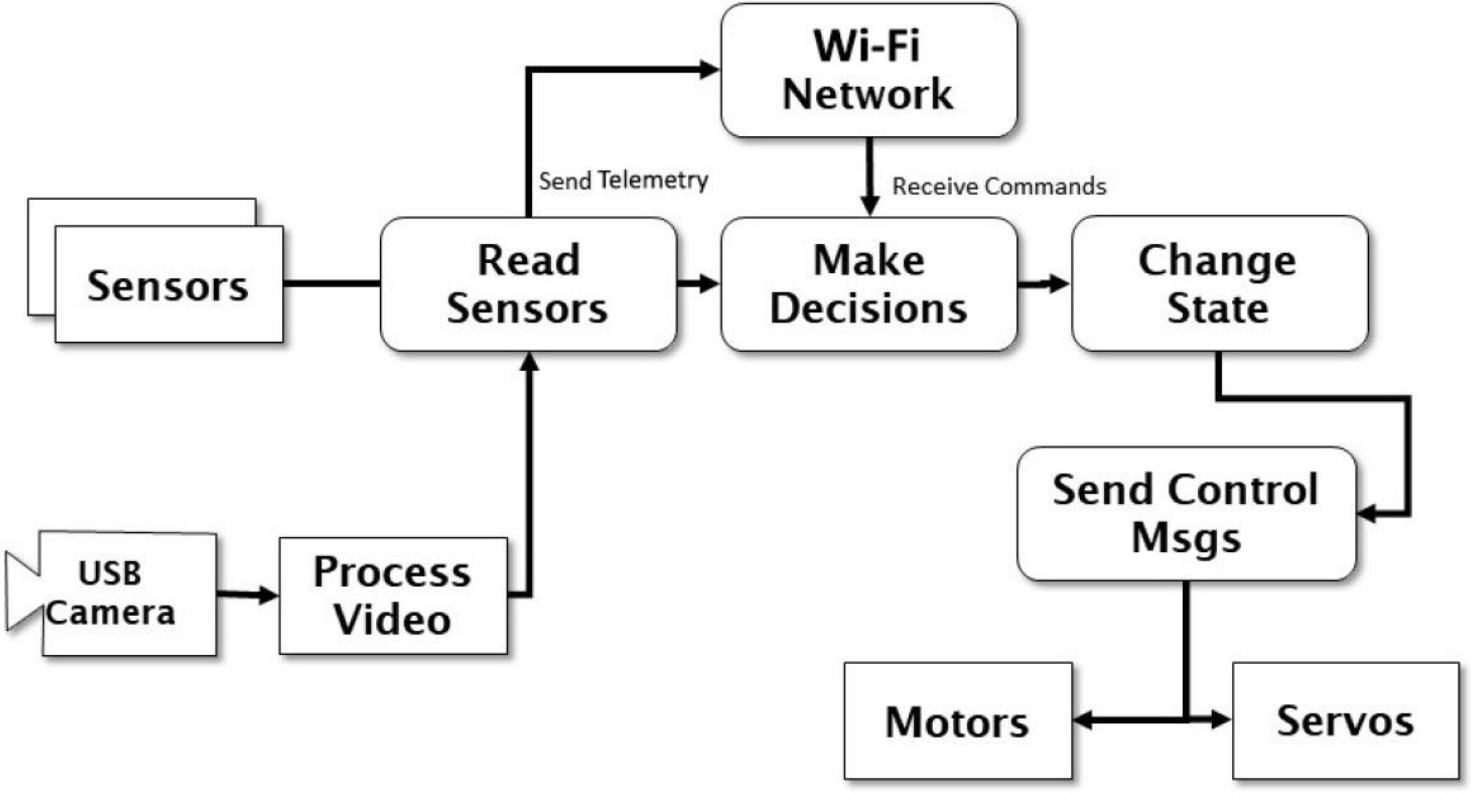




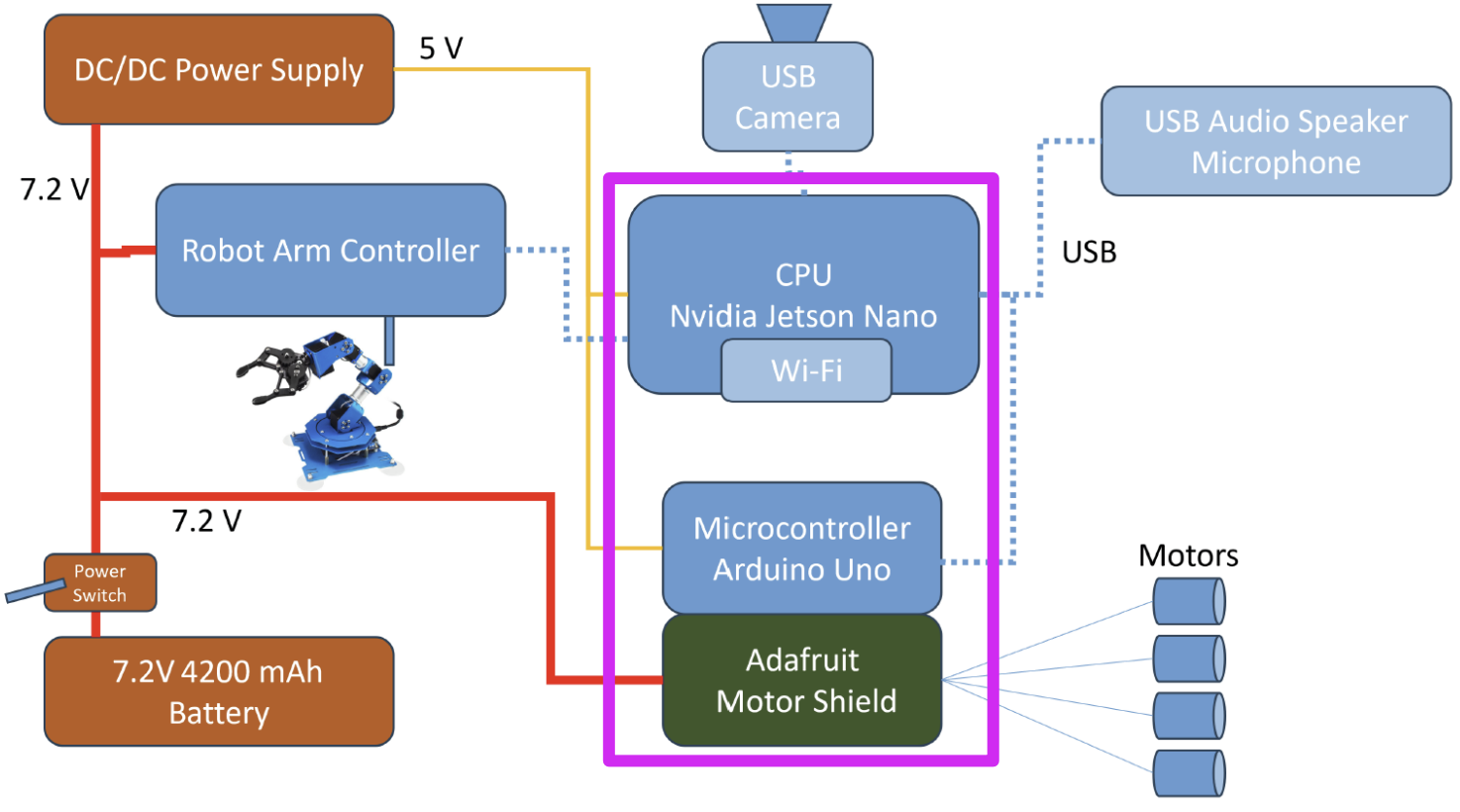

# Robot Mechatronics System (Hardware Components)



# How Robots are Controlled (Action Blocks)



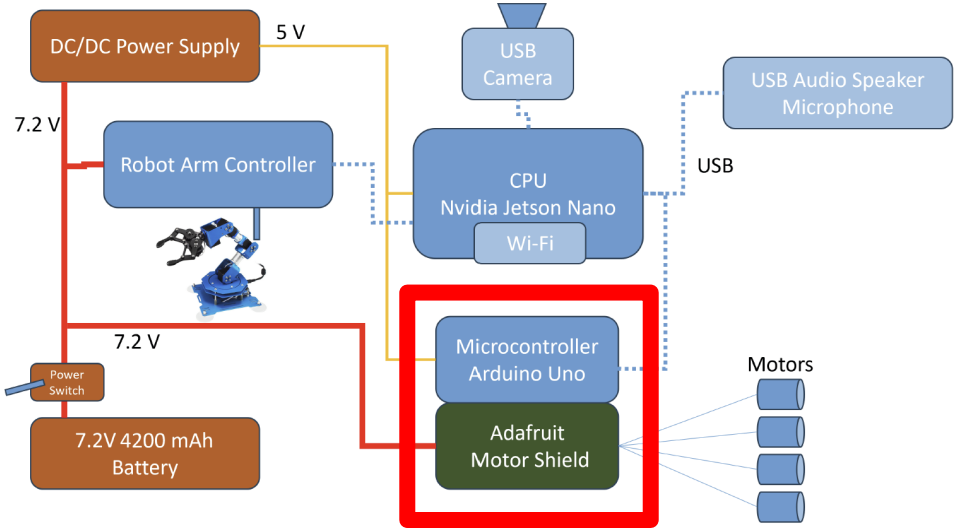
# Usually High-Level (CPU) + Low-Level (µC)

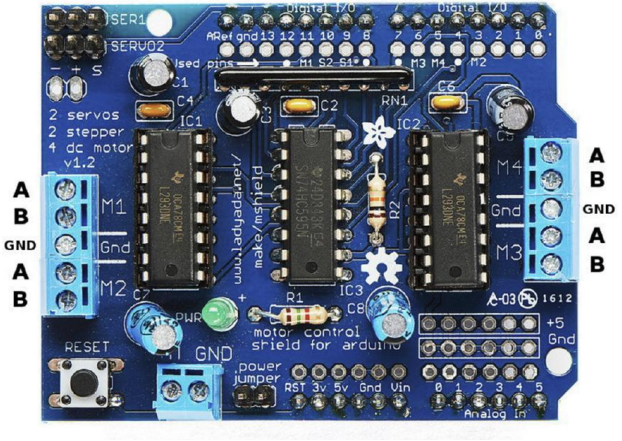


# Microcontroller Level

Key points to note (not our focus)

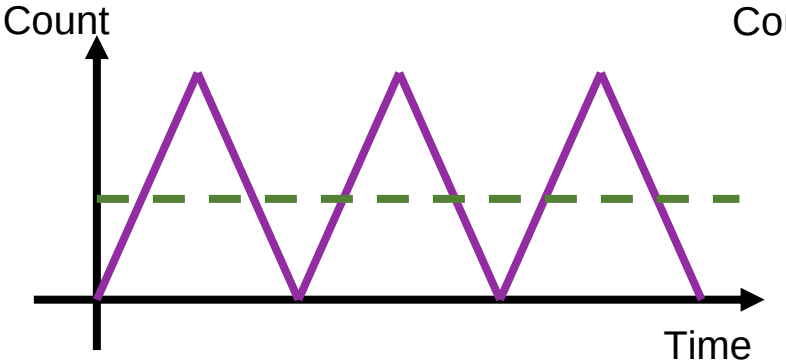
- Single process (sequential)
- Allow for interrupts though
- “Accurate” timing (quartz clock)
- Good for interacting with hardware
- DAC and ADC

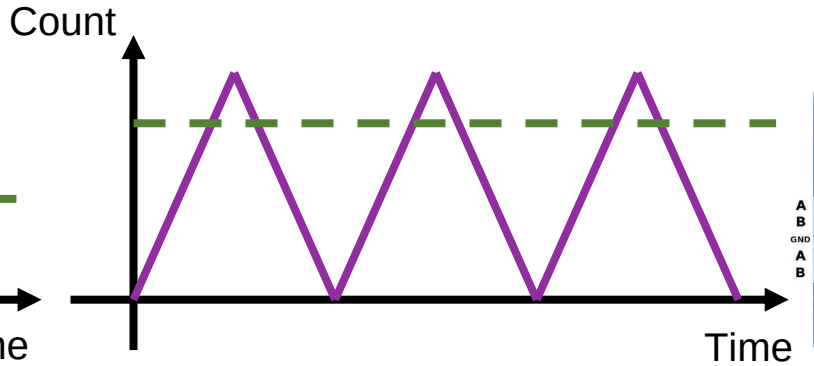


# Digital-Analog and Pulse Width Modulation

Pulse Width Modulation (PWM) is important

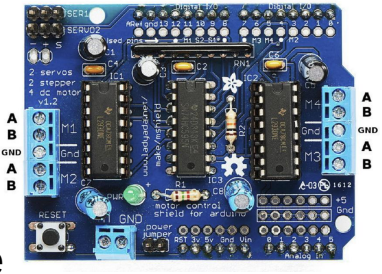





Analog

ADC

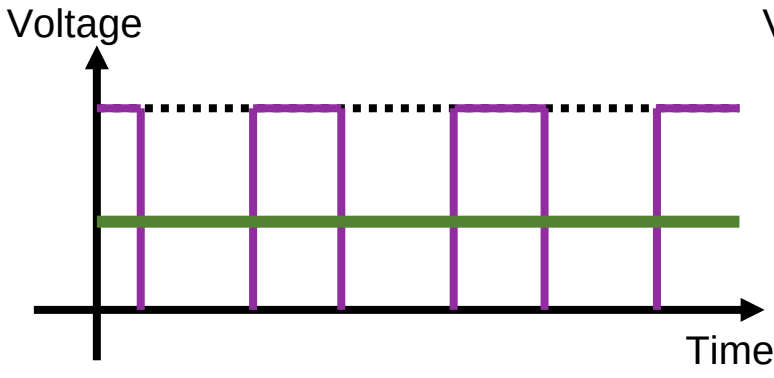


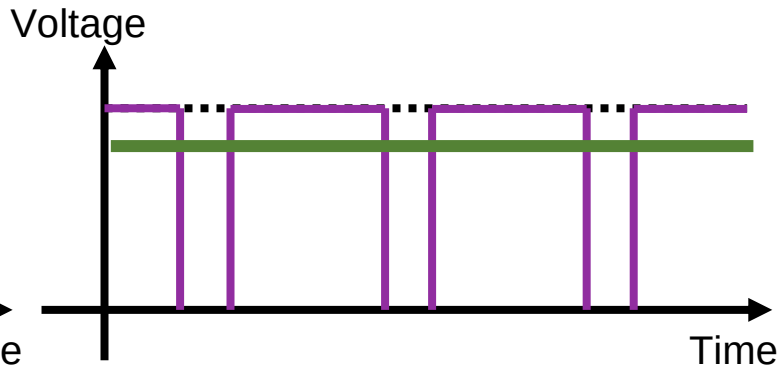
Digital

DAC



Analog



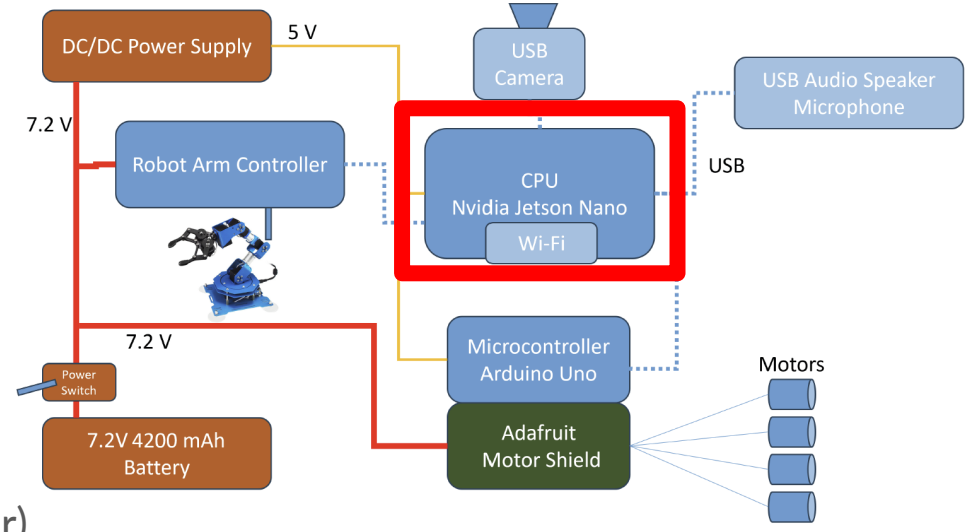



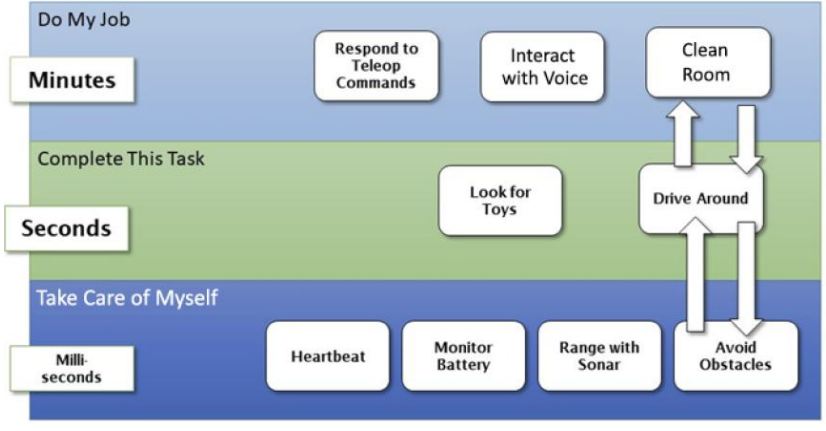
# CPU Level

The brain of the system (our focus)

- Interfacing with sensors (μC)
- Processing sensor data (i.e. vision)
- Interfacing with motors (μC)
- Processing motor commands (controller)
- Planning paths
- Kinematics and dynamics
- Intelligence and decision making

Certainly not a single thread process

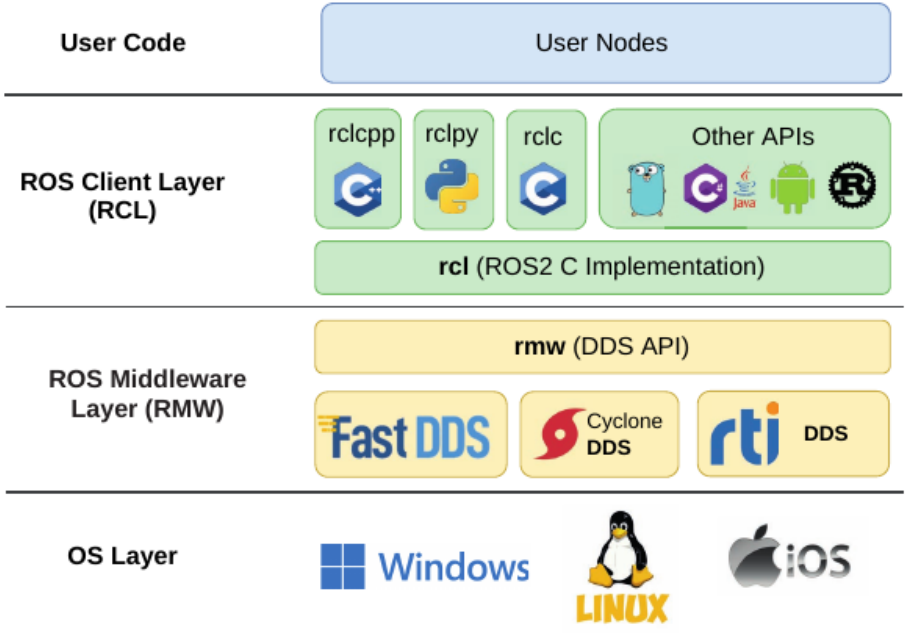


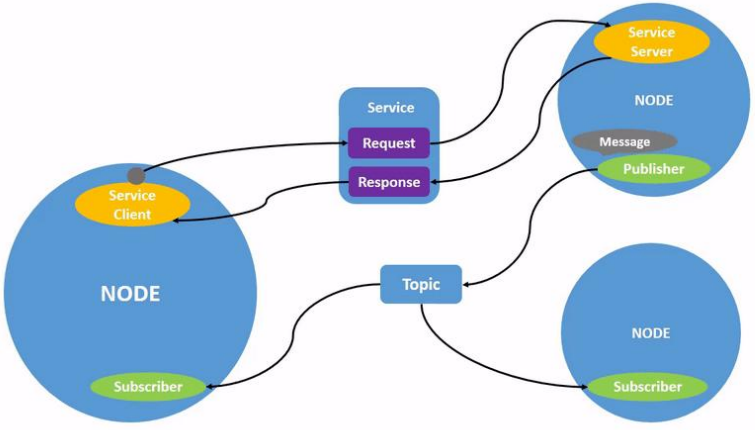



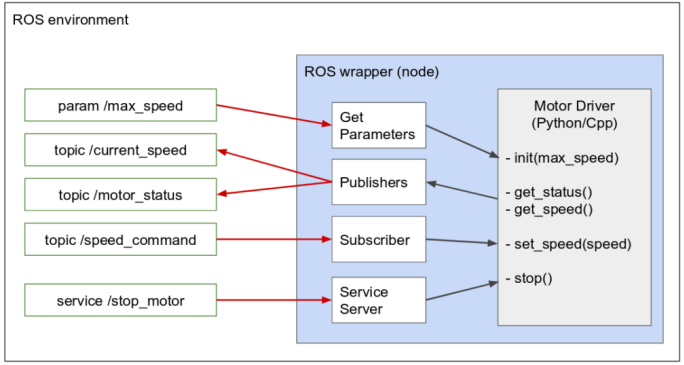
香港中文大學
The Chinese University of Hong Kong

# Robot Operating System

## Why ROS?




Appendix: ROS 2 — 240AR060 - Introduction to ROS

Robot Operating System 2 (ROS 2) Architecture | by Huseyin Kutluca | Software Architecture Foundations | Medium

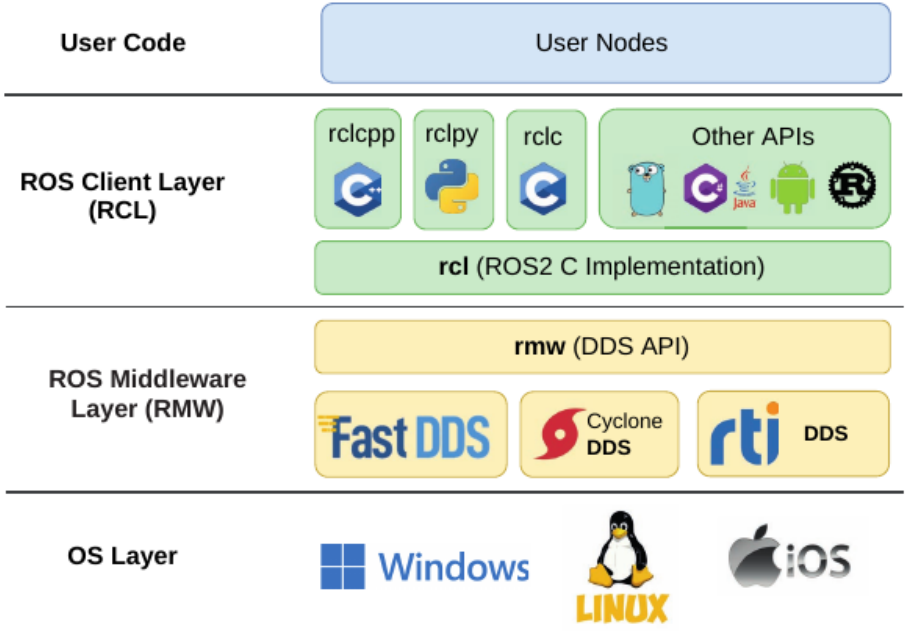
# Robot Operating System

## Why ROS?

- Allows multiple “nodes” (programs)
- Allows multiple computers/OS
- Allows multiple languages
- Back in the olden days…

## What is ROS?

- Not an actual operating system
- It is “middleware”




Appendix: ROS 2 — 240AR060 - Introduction to ROS

# What Do We Need To Know About ROS?

We will teach ROS2, and in particular:

- Setting up ROS2 and nodes within
- The architecture and structure of a node
- Topics and messages
- How to publish and subscribe
- How to run a ROS setup

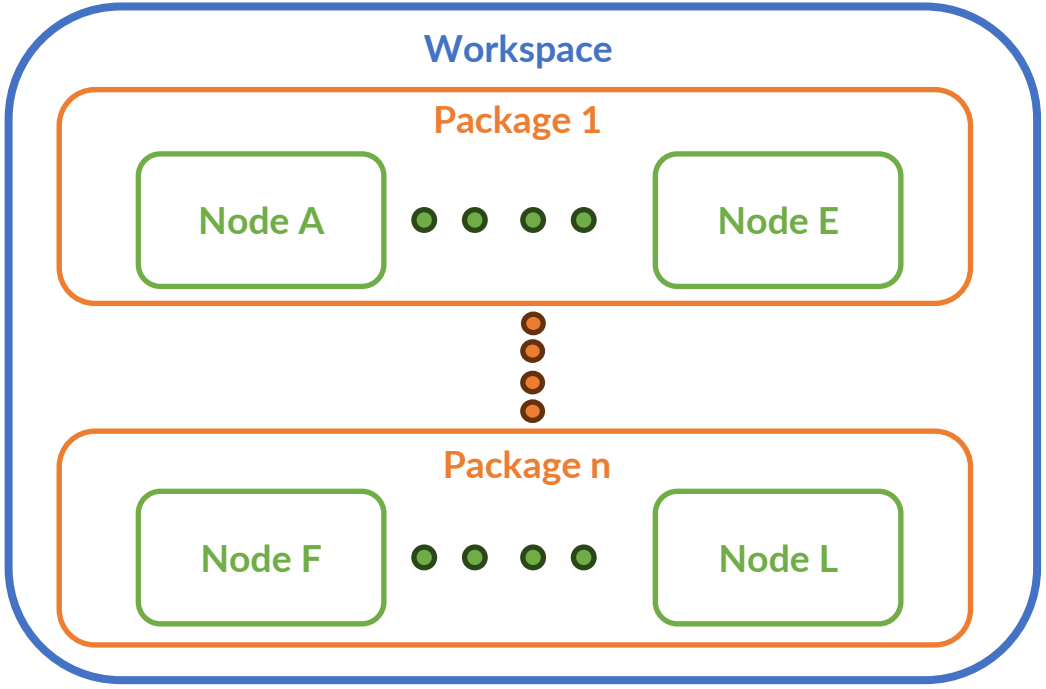
But you need to know Python (or C++)… if not, you will “learn” in your project

# ROS Structure

## Workspace – Package – Node

```
workspace_folder/
    src/
      cpp_package_1/
          CMakeLists.txt
          include/cpp_package_1/
          package.xml
          src/

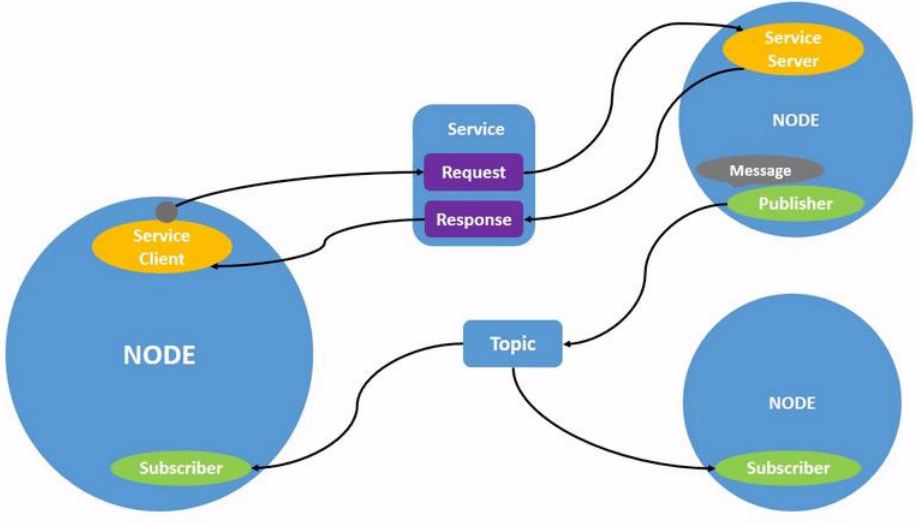
      py_package_1/
          package.xml
          resource/py_package_1
          setup.cfg
          setup.py
          py_package_1/
      ...
      cpp_package_n/
          CMakeLists.txt
          include/cpp_package_n/
          package.xml
          src/
```



# ROS Node

Nodes are individual executables

- Each node in ROS should be responsible for a single, modular purpose, such as:
- e.g. controlling the wheel motors or publishing the sensor data from a laser range-finder
- Each node can send and receive data from other nodes via topics, services, actions, or parameters



# ROS Node

## The “useless” node (Hello World)

```python
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__('my_node_name')

def main(args=None):
    rclpy.init(args=args)
    node = MyNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Main function

Import rclpy and Node

Constructor for class

Create node

Execute thread/timer/callback until kill

Shutdown what is started in “init()”

[Write a Minimal ROS2 Python Node - The Robotics Back-End]

# ROS Node with Timer

## Add a timer (5 Hz) to the node

Difference

```python
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__('my_node_name')
        self.create_timer(0.2, self.timer_callback)

    def timer_callback(self):
        self.get_logger().info("Hello ROS2")

def main(args=None):
    rclpy.init(args=args)
    node = MyNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Create callback that runs every 0.2s (5 Hz)

The “timer_callback” function

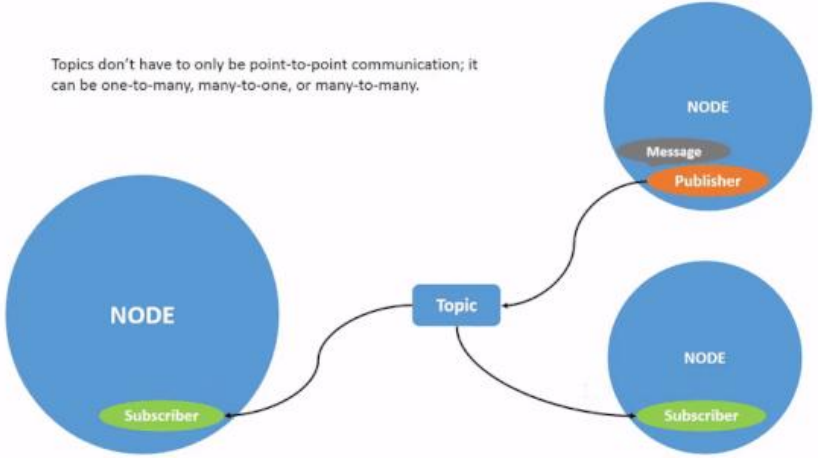
Logger prints to console (debugging)

[Write a Minimal ROS2 Python Node - The Robotics Back-End]

# ROS Messages and Topics

Use topics when you have continuous data streams

- An example of a continuous data stream is laser scan data generated by LIDAR. Data will be continuously published on the topic as long as the robot is up and running.




Messages are used to carry the information within the "topic"

# ROS Publisher and Subscriber

## Publisher sends information to a topic

```python
import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node

from std_msgs.msg import String


class MinimalPublisher(Node):

    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: %d' % self.i
        self.publisher_.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1


def main(args=None):
    try:
        with rclpy.init(args=args):
            minimal_publisher = MinimalPublisher()

            rclpy.spin(minimal_publisher)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass


if __name__ == '__main__':
    main()
```

Create a publisher to send to channel called “topic” of type String
Queue size of 10

Create a message with text data
Publish the message to channel “topic”

# ROS Publisher and Subscriber

## Subscriber listens to information from a topic

```python
import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node

from std_msgs.msg import String


class MinimalSubscriber(Node):

    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic',
            self.listener_callback,
            10)
        self.subscription  # prevent unused variable warning

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)


def main(args=None):
    try:
        with rclpy.init(args=args):
            minimal_subscriber = MinimalSubscriber()

            rclpy.spin(minimal_subscriber)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass


if __name__ == '__main__':
    main()
```

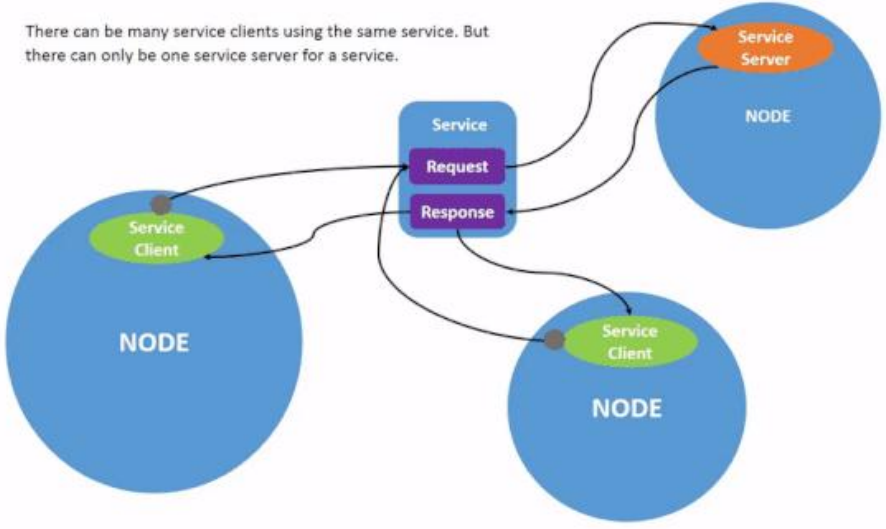
Set up subscriber on listen to channel “topic” (same name as publisher)
Message type is String
When message is received then run “listener_callback()”

Callback function
Hence no timer is needed (asynchronous)

# ROS Service and Client

Services should be used for fast request/response communication

- Like a function call across nodes
- E.g.: Call a service to load a new map once a robot has reached a new floor of a building
- E.g.: Call a service to compute inverse kinematics



# ROS Service and Client

## First setup the service node

```python
from example_interfaces.srv import AddTwoInts

import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node


class MinimalService(Node):

    def __init__(self):
        super().__init__('minimal_service')
        self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.add_two_ints_callback)

    def add_two_ints_callback(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info('Incoming request\na: %d b: %d' % (request.a, request.b))

        return response


def main():
    try:
        with rclpy.init():
            minimal_service = MinimalService()

            rclpy.spin(minimal_service)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass


if __name__ == '__main__':
    main()
```

**AddTwoInts.srv**

```
int64 a
int64 b
---
int64 sum
```

Create a service called “add_two_ints”
With callback “add_two_ints_callback()” of type AddTwoInts

Callback function (inputs “a” and “b”, output “sum”)

# ROS Service and Client

## Then the client

```python
from example_interfaces.srv import AddTwoInts

import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node


class MinimalClientAsync(Node):

    def __init__(self):
        super().__init__('minimal_client_async')
        self.cli = self.create_client(AddTwoInts, 'add_two_ints')
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('service not available, waiting again...')
        self.req = AddTwoInts.Request()

    def send_request(self):
        self.req.a = 41
        self.req.b = 1
        return self.cli.call_async(self.req)


def main(args=None):
    try:
        with rclpy.init(args=args):
            minimal_client = MinimalClientAsync()
            future = minimal_client.send_request()
            rclpy.spin_until_future_complete(minimal_client, future)
            response = future.result()
            minimal_client.get_logger().info(
                'Result of add_two_ints: for %d + %d = %d' %
                (minimal_client.req.a, minimal_client.req.b, response.sum))
    except (KeyboardInterrupt, ExternalShutdownException):
        pass


if __name__ == '__main__':
    main()
```

Set up client on name “add_two_ints” with type AddTwoInts

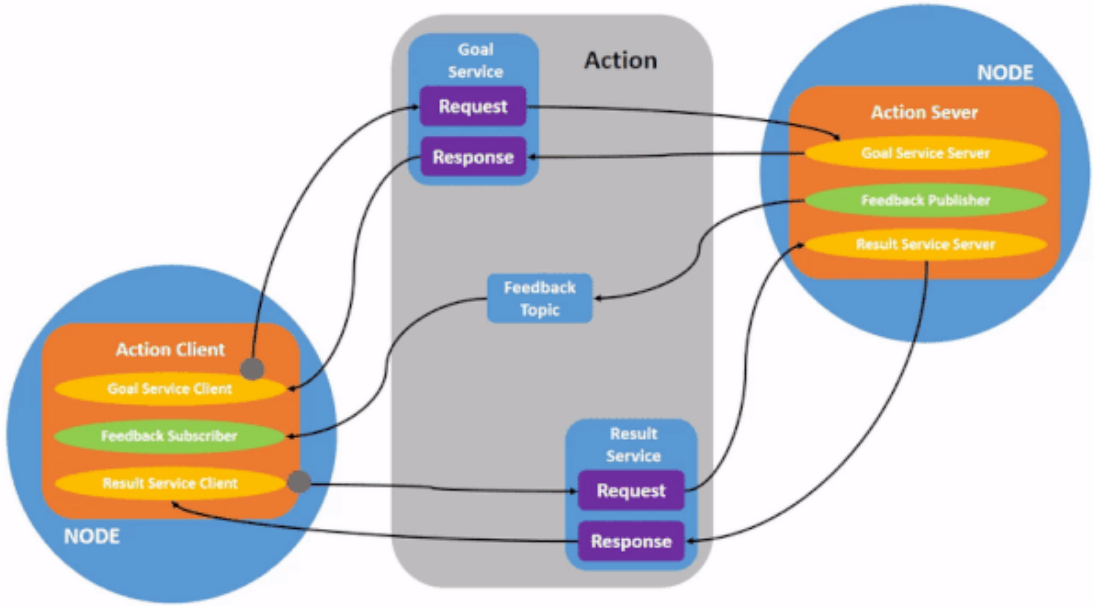
Define the request (sample)

Call the request

# ROS Actions

Actions should be used for long running processes that have a clear beginning and end

- An action is made up of a goal, interim feedback (to keep you updated on the progress being made to reach the goal), and a result.



# Common Message Types

`std_msgs`:
Contains fundamental, generic message types.

- `std_msgs/msg/String`: A simple string.
- `std_msgs/msg/Int32`: A 32-bit integer.
- `std_msgs/msg/Bool`: A boolean value.
- `std_msgs/msg/Header`: Contains a timestamp and frame ID, commonly used as the first field in other messages for time synchronization and coordinate frame information.

`geometry_msgs`:
Defines messages for geometric primitives and transformations.

- `geometry_msgs/msg/Point`: A 3D point (x, y, z).
- `geometry_msgs/msg/Vector3`: A 3D vector (x, y, z).
- `geometry_msgs/msg/Quaternion`: A quaternion for representing 3D rotations (x, y, z, w).
- `geometry_msgs/msg/Pose`: A position and orientation (Point and Quaternion).
- `geometry_msgs/msg/Twist`: Linear and angular velocities (Vector3 for each).
- `geometry_msgs/msg/Transform`: A translation and rotation.

`sensor_msgs`:
Used for various sensor data.

- `sensor_msgs/msg/Image`: Raw or compressed image data.
- `sensor_msgs/msg/PointCloud2`: A 3D point cloud.
- `sensor_msgs/msg/LaserScan`: Data from a 2D laser scanner.
- `sensor_msgs/msg/Imu`: Inertial Measurement Unit data (orientation, angular velocity, linear acceleration).

`nav_msgs`:
Messages related to navigation and mapping.

- `nav_msgs/msg/Odometry`: Robot pose and velocity in a free-space coordinate frame.
- `nav_msgs/msg/Path`: A sequence of poses, representing a path.

`diagnostic_msgs`:
Messages for system diagnostics and status.

- `diagnostic_msgs/msg/DiagnosticStatus`: Status of a diagnostic task.
- `diagnostic_msgs/msg/DiagnosticArray`: An array of DiagnosticStatus messages.

# Custom Messages and Server Types

## Very easy to create also

### Custom Message

In the `tutorial_interfaces/msg` directory you just created, make a new file called `Num.msg` with one line of code declaring its data structure:

```
int64 num
```

This is a custom message that transfers a single 64-bit integer called `num`.

Also in the `tutorial_interfaces/msg` directory you just created, make a new file called `Sphere.msg` with the following content:

```
geometry_msgs/Point center
float64 radius
```

### Custom Server

Back in the `tutorial_interfaces/srv` directory you just created, make a new file called `AddThreeInts.srv` with the following request and response structure:

```
int64 a
int64 b
int64 c
---
int64 sum
```

**Request**

**Response**

# Other ROS Things to Learn/Know About ROS

There are quite a few details to read, for example:

- Using “colcon” to build packages

```
$ colcon build --symlink-install
```

Run this at the root of the workspace

- ros2 run

With the environment sourced, we can run executables built by colcon. Let’s run a subscriber node from the examples:

```
$ ros2 run examples_rclcpp_minimal_subscriber subscriber_member_function
```

- ros2 launch

```
from launch import LaunchDescription
import launch_ros.actions


def generate_launch_description():
    return LaunchDescription([
        launch_ros.actions.Node(
            namespace='turtlesim1', package='turtlesim',
            executable='turtlesim_node', output='screen'),
        launch_ros.actions.Node(
            namespace='turtlesim2', package='turtlesim',
            executable='turtlesim_node', output='screen'),
    ])
```

```
$ ros2 launch turtlesim multisim.launch.py
```

# Other ROS Things to Learn/Know About ROS

## Setting up the package and nodes

- Create package

Recall that packages should be created in the `src` directory, not the root of the workspace. So, navigate into `ros2_ws/src`, and run the package creation command:

```
$ ros2 pkg create --build-type ament_python --license Apache-2.0 py_pubsub
```

- package.xml

```
<description>Examples of minimal publisher/subscriber using rclpy</description>
<maintainer email="you@email.com">Your Name</maintainer>
<license>Apache-2.0</license>

<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

# Other ROS Things to Learn/Know About ROS

## Setting up the package and nodes

- setup.py

```
maintainer='YourName',
maintainer_email='you@email.com',
description='Examples of minimal publisher/subscriber using rclpy',
license='Apache-2.0',

entry_points={
        'console_scripts': [
                'talker = py_pubsub.publisher_member_function:main',
                'listener = py_pubsub.subscriber_member_function:main',
        ],
},
```

- setup.cfg

```
[develop]
script_dir=$base/lib/py_pubsub
[install]
install_scripts=$base/lib/py_pubsub
```

# Some Additional ROS2 Points

Note the following

- Constantly evolving, ROS1 to ROS2 and multiple versions
- Reading is not the same as “trying”, set one up yourself
- Lots of “details” once you get into it, it’s a very vast framework
- Lots of examples online to read, including https://docs.ros.org/en